

Concretely-Efficient Multi-Key Homomorphic Secret Sharing and Applications

Kaiwen He¹, Sacha Servan-Schreiber², Geoffroy Couteau³, and Srinivas Devadas¹

¹ MIT

² Tinfoil

³ Université Paris Cité, CNRS, IRIF

Abstract. Homomorphic secret sharing (HSS) is a powerful cryptographic primitive that enables efficient, low-communication secure computation without the use of fully homomorphic encryption. Public-key HSS is a well-known variant that supports inputs from multiple parties, but all parties must agree on a joint public key *before* any party can encode their inputs, requiring extra rounds of communication in applications. Recently, Couteau et al. (EUROCRYPT 2025) constructed multi-key HSS (MKHSS)—a new primitive which allows parties to encode their inputs under independent keys—under the DCR assumption. MKHSS assumes only a reusable common reference string, without the need for prior interactions between parties or a public-key infrastructure.

In this paper, we construct and implement the first concretely-efficient MKHSS scheme under the same assumptions used by Couteau et al. Using an algorithmic insight that reduces the largest modulus in Couteau et al. from N^4 to N^2 , our optimized implementation can homomorphically multiply inputs in 5.0 milliseconds—while an implementation of Couteau et al. requires 224.6 milliseconds—thereby achieving a $45\times$ speedup.

A powerful application of MKHSS is to realize attribute-based non-interactive key exchange (ANIKE), which generalizes password-authenticated key exchange (PAKE) to arbitrary attribute policies. ANIKE is currently only known from MKHSS and in this paper we show the first practical instantiation of ANIKE for two concrete predicate types with applications to geolocation-based key exchange and *fuzzy* PAKE.

We use our implementation to evaluate the first concretely-efficient ANIKE schemes for a range of practically useful policies. Using our implementation, two parties can perform a geolocation-based key exchange in under one second and a fuzzy PAKE on an 8-word passphrase in a few seconds for realistic parameters, on a single core, achieving a roughly $30\times$ speedup over Couteau et al. for both applications.

Table of Contents

1	Introduction	2
1.1	An Application of MKHSS	3
1.2	Our Contributions	3
2	Technical Overview: MKHSS	4
2.1	Notation	4
2.2	Prior Work	4
2.3	Our Optimizations	6
2.4	Correctness	7
3	Technical Overview: ANIKE	7
3.1	Our Contributions	8
3.2	Starting Point: Couteau et al.	9
3.3	Making Generic ANIKE Efficient	9
3.4	Geolocation-Based Key Exchange	10
3.5	Fuzzy PAKE	12
4	Preliminaries	13
4.1	Assumptions	13
4.2	Damgård-Jurik-ElGamal Encryption	14
4.3	Distributed Discrete Log	17
4.4	Non-Interactive Multiplication	17
5	Definitions	19
5.1	Multi-Key HSS	19
5.2	Attribute-Based Non-Interactive Key Exchange	20
6	Full Multi-Key HSS Construction	22
7	Full ANIKE Construction	26
8	Evaluation	27
8.1	Experimental Setup	27
8.2	Multi-Key HSS	28
8.3	Application: Geolocation-Based Key Exchange	30
8.4	Application: Fuzzy PAKE	32
8.5	Further Acceleration Using Vectorization	35
9	Related Works	35
10	Conclusion	35
A	Malicious Security	39

1 Introduction

Homomorphic secret sharing (HSS) [BGI16] allows two or more parties, each holding a public encoding of some secret input x , to locally compute a function f on the input and obtain a subtractive secret share of $f(x)$. It provides an efficient, distributed alternative to fully-homomorphic encryption (FHE) [Gen09] for secure computation.

The most basic form of HSS only supports computation over private input provided by a single party. The well-known variant, called public-key HSS [BGI17, BCG⁺17, ADOS22], supports inputs by multiple parties. However, all public-key HSS schemes require that all parties agree on some joint public key *before* they can even generate the input encodings, which are typically ciphertexts encrypted under the joint key. In addition, the parties who wish to evaluate a function need to obtain correlated evaluation keys, which can be seen as additive shares of the joint secret key. This requirement often requires an extra round of communication in many applications.

To address these shortcomings, Couteau et al. [CDH⁺25] introduced a new primitive called multi-key HSS (MKHSS): using only a common reference string (CRS), each party can generate their own public and secret key pair, *and* generate input encodings under its own key, without interaction with the other party. Each party can then exchange the concatenation of their public key and input encodings in a simultaneous message, and finally locally compute a common function.

We stress that the CRS can be generated once and used in multiple instances of the MKHSS protocol. Furthermore, the public keys and input encodings can both be reused across multiple executions of the protocol, potentially between *different* parties. The reusability of the CRS and public encodings is a significant advantage of MKHSS over public-key HSS, where the correlated evaluation keys are tied to the joint secret key and not reusable. The CRS can even be generated using a secure multi-party computation protocol, which will remove the trust on a single CRS generator, while maintaining protocol efficiency since this only need to be done once.

While the notion of MKHSS is promising, Couteau et al. [CDH⁺25] only provided a *theoretical* construction for MKHSS. It performs poorly in practice due to the need to use a modulus that is *four* times as large as an RSA modulus for the same level of security. We uncover algorithmic insights and overcome this major obstacle. In the end, our final construction only requires a modulus of *half* the size, matching the modulus size of the Paillier cryptosystem [Pai99], a simple cryptographic construction from the DCR assumption.

The reduced modulus size allows our scheme to achieve a significant concrete improvement in the running time of every procedure as well as savings in communication and local storage. To give an example, a single homomorphic multiplication (the bottleneck in the key exchange applications) takes 5.0 milliseconds, a 45 $\times$ speedup compared to 224.6 milliseconds obtained by our implementation of the theoretical construction of Couteau et al.

1.1 An Application of MKHSS

Attribute-based non-interactive key exchange (ANIKE) is a new primitive that allows two parties—each holding some private input—to agree on a common key if and only if their inputs satisfy some policy predicate, and does so without revealing to each other any information beyond whether the predicate is satisfied.

Moreover, ANIKE has the same communication pattern as a non-interactive key exchange (NIKE) [FHKP13], e.g., Diffie-Hellman [DH76]. This property distinguishes it from all prior works on interactive key exchange [DHP⁺18, KKL⁺16, BFH⁺23] for practically interesting policies.

ANIKE generalizes the notion of PAKE [BM92] and even fuzzy PAKE [DHP⁺18, BFH⁺23]. Couteau et al. [CDH⁺25] provided the first *theoretical* construction of ANIKE for all HSS-computable policies by using MKHSS as a black box.⁴

However, their construction is theoretical for more than one reason. Even if the underlying MKHSS scheme is made practical, they require evaluating a pseudorandom function (PRF) *homomorphically* over the encoded inputs, which is prohibitively expensive. We provide a practical alternative construction that only requires hashing *outside of* HSS. Our implementation exploits a property specific to our optimized scheme (and Couteau et al.) to unlock performance gains, but can still be made concretely-efficient with black-box use of a generic MKHSS construction.

We show that our ANIKE can realize predicates with concrete applications: geolocation-based key exchange and fuzzy password-authenticated key exchange (PAKE).

Importance of Non-interactivity. Public-key cryptography is predicated on the existence of non-interactivity: parties can post their keys on the internet and anyone can non-interactively encrypt a message under that key. In contrast, alternative MPC based approaches to key exchange [DHP⁺18, KKL⁺16, BFH⁺23] break this model since both parties need to be *simultaneously online* to derive a common key, which is incompatible with the modern client-server setting [HLP11].

1.2 Our Contributions

To summarize, our contributions are:

1. Several insightful algorithmic optimizations to MKHSS that critically improve performance.
2. Identifying two concretely-practical and useful applications of MKHSS: geolocation-based key exchange and fuzzy PAKE based on passphrases.
3. A concretely-efficient, open-source C++ implementation of multi-key HSS, a recent primitive, as well as its two applications to key exchange.⁵

⁴ Existing HSS schemes are sufficiently powerful to compute polynomial-size branching programs, which capture all functions in NC¹.

⁵ Open-source code is available at: <https://github.com/kevin-he-01/mkhss/>.

2 Technical Overview: MKHSS

A homomorphic secret sharing (HSS) scheme typically supports a computation model known as *restricted multiplication straight-line* (RMS) programs, which homomorphically operates on B -bounded integer values of two types: input and memory values. A program starts with an initial memory value of 1 and a list of input values, and is capable of performing the following operations:

1. Linear operation on values of the same type.
2. Multiplication between an input value and a memory value, producing a memory value (RMS multiplication).

The starting point of our construction is the construction of MKHSS from the decisional composite residuosity (DCR) assumption [CDH⁺25]. To achieve practical performance gains, we rely on an assumption called the short-exponent indistinguishability (SEI) assumption [KK04, ABC⁺24], which is assumed by Couteau et al. [CDH⁺25] when providing concrete running time estimates for MKHSS. Over a cyclic group $\mathbb{G}$ generated by g , SEI says that it is infeasible to distinguish between (g, g') and (g, g^r) , where $g' \leftarrow \$ \mathbb{G}$ is a random group element, and $r \leftarrow \$ \{0, \dots, 2^{2\lambda}\}$ is the *short* exponent.

Security of SEI. It was proven in [ABC⁺24, Appendix A] that SEI is equivalent to the standard short-exponent discrete logarithm (SEDL) assumption, which says that it is infeasible to *find* the short exponent $r \in \{0, \dots, 2^{2\lambda}\}$ given (g, g^r) . Pollard [Pol78] introduced SEDL in 1978 (merely two years after Diffie-Hellman [DH76]) and provided Pollard’s lambda algorithm, which remains the best cryptanalysis to our knowledge, and justifies our exponent bound of $2^{2\lambda}$ for security parameter λ . Since then, many prior works (e.g., [CKLR21, CGKR22]) on applied and theoretical cryptography assume SEI or SEDL.

In addition to being a decades-old, well-established assumption, SEI and SEDL are often assumed in practice. Internet standards for TLS [Gil16] and SSH [FPS06] both suggest the use of short exponents for performance reasons. A 2016 study [VAS⁺17] reports that eight out of nine popular cryptographic libraries (most notably OpenSSL) *allow* the use of short exponents, and *deployed* applications (e.g., Lighttpd and Unbound) use short exponents.

2.1 Notation

We let $a \wedge b$ and $a \vee b$ denote logical conjunction (AND) and disjunction (OR), respectively. For a logical proposition P , e.g., $x < y$, define $\mathbb{1}(P) = 1$ if P is true and 0 otherwise.

Denote $[m] := \{0, 1, \dots, m-1\}$, $\mathbb{Z}_m$ as the additive group of integers modulo m , and $\mathbb{Z}_m^*$ as the multiplicative group of integers coprime with m modulo m . A boldface variable $\mathbf{x} := (x_1, x_2, \dots, x_L)$ denotes a vector, and an uppercase boldface letter $\mathbf{A}$ denotes a matrix or higher-dimensional tensor. $\langle y \rangle = (\langle y \rangle_A, \langle y \rangle_B)$ denotes a subtractive secret share of y between two parties (called Alice and Bob), such that $\langle y \rangle_A - \langle y \rangle_B = y$ over the *integers*. The notation extends to vectors and matrices by element-wise secret sharing. $\sigma \in \{A, B\}$ denotes the identity of some arbitrary party, and $1 - \sigma$ denotes the *other* party that is not σ . We let s denote the secret key of Paillier-ElGamal encryption [BCP03], which we use to instantiate HSS.

2.2 Prior Work

2.2.1 HSS from Paillier-ElGamal We describe a key idea to achieve single-key two-party HSS for RMS programs based on Paillier-ElGamal encryption [BCP03]. This is similar to [OSY21], but simplified using short exponents.

We first show how to secret share memory and input values in an RMS program:

1. *Memory share*: Denoted $\langle\!\langle x \rangle\!\rangle_\sigma$. It is defined as $\langle\!\langle x \rangle\!\rangle_\sigma = (\langle x \cdot s \rangle_\sigma, \langle x \rangle_\sigma)$, a pair of subtractive shares.
2. *Input share*: Denoted $\llbracket x \rrbracket_\sigma$. It is generated by sampling $r, r' \leftarrow \$ [2^{2\lambda}]$ and computing

$$\begin{aligned} \llbracket x \rrbracket_\sigma &= ((\text{ct}_0, \text{ct}_1), (\text{ct}'_0, \text{ct}'_1)) \\ &= ((g^r(1+N)^x, g^{-rs}), (g^{r'}, g^{-r's}(1+N)^x)). \end{aligned}$$

All Paillier-based [Pai99] schemes use the observation that $(1+N)^x \equiv 1 + Nx \pmod{N^2}$ due to the binomial theorem, and therefore, the base- $(1+N)$ discrete log of $z \in \mathbb{Z}_{N^2}^*$ is $(z-1)/N$.

A ciphertext in Paillier-ElGamal is a pair of $\mathbb{Z}_{N^2}^*$ elements (c_0, c_1) . To decrypt, we compute $\text{DLog}_{1+N}(c_0^s c_1) = (c_0^s c_1 - 1)/N$. Regarding the components of $\llbracket x \rrbracket_\sigma$, one may verify that $(\text{ct}_0, \text{ct}_1)$ decrypts to xs and $(\text{ct}'_0, \text{ct}'_1)$ decrypts to x .

Using the linearity of subtractive shares and additive homomorphism of Paillier-ElGamal, it is trivial to perform addition, subtraction, and scalar multiplications on shares of the same type. It remains to show how to do the RMS multiplication: obtain $\langle xy \rangle_\sigma$, given $\langle y \rangle_\sigma$ and $\llbracket x \rrbracket_\sigma$.

Let's assume for now that somehow we have y and s in the clear, so we can transform $(\text{ct}_0, \text{ct}_1)$ —an encryption of x —into an encryption of xy via component-wise exponentiation by y (using additive homomorphism of Paillier-ElGamal), and then decrypt using s . When written down, this looks like $\text{DLog}_{N+1}(\text{ct}_0^{ys} \cdot \text{ct}_1^y)$.

Observe that the combined multiply-and-decrypt procedure uses ys and y . Now, consider what happens if we only have secret shares of them, which is exactly how the memory share is structured. Let $(\text{ct}_0, \text{ct}_1) = (g^{r'}, g^{-r's}(1+N)^x)$, then $\text{ct}_0^{\langle y \rangle_\sigma} \text{ct}_1^{\langle y \rangle_\sigma} = g^{\langle 0 \rangle_\sigma} (1+N)^{\langle xy \rangle_\sigma}$. It might be tempting to think that discrete log with respect to $N+1$ can recover $\langle xy \rangle_\sigma \bmod N$. However, note that the definition of a subtractive share only guarantees that $\langle 0 \rangle_A - \langle 0 \rangle_B = 0$, and in general $\langle 0 \rangle_\sigma \neq 0$. Fortunately, using the distributed discrete log (DDLog) procedure [OSY21, Algorithm 3.2], party σ can recover a value z_σ such that $z_A - z_B \equiv xy \bmod N$, so z_σ is a valid subtractive share of xy with overwhelming probability when $|xy| \ll N$. Specifically, if the RMS program is B -bounded for some $B \leq 2^{-\tau}N$, the failure probability of this procedure is upper bounded by $2^{-\tau}$.

Provided that $|xys| \ll N$ (this is where short exponent s is useful), we can use the same procedure to obtain $\langle xys \rangle_\sigma$ from the encryption of xs : $(g^r(1+N)^x, g^{-rs})$. At the end, this gives a procedure to obtain $\langle xy \rangle_\sigma = (\langle xys \rangle_\sigma, \langle xy \rangle_\sigma)$ from $\llbracket x \rrbracket_\sigma$ and $\langle y \rangle_\sigma$.

2.2.2 Share Synchronization to Achieve MKHSS Recall that MKHSS allows two parties to generate independent public-secret key pairs, and each party creates input shares as encryptions under its own key. The problem comes when evaluating functions on such inputs, which could be supplied by either party and thus encrypted under different keys.

The approach taken by Couteau et al. [CDH⁺25] to achieve MKHSS is the following: During the evaluation procedure, first transform all encryptions in the input shares to be under some joint public key (this step is referred to as share *synchronization*), then use a procedure to bootstrap the first memory share of 1 (which contains a subtractive secret share of the joint secret key), and finally, evaluate the function.

Specifically, denote s_σ as the secret key of party σ , and the corresponding public key as $f_\sigma = g^{-s_\sigma}$. We define the joint secret key as $s = s_A s_B$. Observe that it is trivial for each party to then obtain the joint public key via a Diffie-Hellman key exchange, e.g., Alice computes $f = f_A^{s_B}$. In addition, it turns out that encryptions of xs_A under the public key f_A can be transformed into encryptions of xs under the *joint* public key f easily, via exponentiating the second component by s_B :

$$(\text{ct}_{0,A}, \text{ct}_{1,A}^{s_B}) = (g^r(1+N)^x, f_A^{-rs_B}) = (g^r(1+N)^x, f^{-r}).$$

Unfortunately, doing the same thing for encryptions of x results in an encryption of $-s_B x$:

$$(g^r, (f_A^{-r}(1+N)^x)^{-s_B}) = (g^r, f^{-r}(1+N)^{-s_B x}).$$

Couteau et al. [CDH⁺25] solved this by requiring both parties to sample the individual secret key to be of a special form: $s_\sigma = t_\sigma \cdot N + 1$. Observe that $s_\sigma \equiv 1 \bmod N$, and since the order of $N+1$ is N , $(1+N)^{-s_B x} \equiv (1+N)^x \bmod N^2$. It turns out that if we sample $t_\sigma \leftarrow \mathbb{S}[2^{2\lambda}]$, this remains secure under SEI. It turns out such secret key structure does not harm security. A curious reader may jump ahead to our proof of Theorem 14, specifically Equation (3).

A significant problem with this approach is that even when t_σ is short, the structure of s_σ means its size must be as large as the modulus N . To make matters worse, the joint secret key is a product $s = s_A s_B$, so it is even bigger. Recall that we require that after a multiplication, $xys \ll N$. This is impossible to satisfy regardless of the size of N .

Couteau et al. used a heavyweight solution to this problem: they encrypted xs using Damgård-Jurik-ElGamal (DJEG)—a generalization of Paillier-ElGamal with the ciphertext modulus being N^{w+1} instead of N^2 . They specifically used the parameter $w = 3$, thereby increasing the ciphertext modulus from N^2 to N^4 , adding significant computation overhead to many operations. Since the

order of $1 + N$ is N^3 in $\mathbb{Z}_{N^4}^*$, we now only require $xy s \ll N^3$. A further complication is that to ensure encryptions of x can still be synchronized, Paillier-ElGamal is used *just* for it. Even when the asymptotically efficient Karatsuba [KO62] multiplication algorithm is used, any group operation in N^4 is still at least $3\times$ slower than that in N^2 .

The last step is to bootstrap the initial memory share $\langle 1 \rangle_\sigma = (\langle s \rangle_\sigma, \langle 1 \rangle_\sigma)$, specifically $\langle s \rangle_\sigma$. This can be done using a procedure called Non-Interactive Multiplication (NIM) [BDSS25, CDH⁺25].

2.3 Our Optimizations

We uncover several algorithmic insights to significantly improve the performance of MKHSS, including overcoming a major obstacle in Couteau et al., i.e., the need to use the modulus N^4 . The largest modulus in our final construction is N^2 , the same modulus used by Paillier’s cryptosystem, and is optimal for constructions based on the DCR assumption (which uses the group $\mathbb{Z}_{N^2}^*$).

The reduced modulus size allows us to achieve a significant concrete improvement in both running time (due to faster group operations) and reduction in communication and memory usage (due to the smaller size of integers).

2.3.1 Deriving $\langle y \rangle_\sigma$ directly from $\langle ys \rangle_\sigma$ Consider the RMS multiplication procedure, which takes in $\llbracket x \rrbracket_\sigma$ and $\langle y' \rangle_\sigma$ and produces a memory share of a product $y = xy'$: $\langle y \rangle_\sigma = (\langle y \rangle_\sigma, \langle ys \rangle_\sigma)$. In Couteau et al., they derive $\langle y \rangle_\sigma$ and $\langle ys \rangle_\sigma$ separately. Computing *either* of the two requires two modular exponentiations, so the total cost to compute $\langle y \rangle_\sigma$ is dominated by four exponentiations.

Instead, we derive $\langle y \rangle_\sigma$ directly from $\langle ys \rangle_\sigma$ *without* any exponentiations, reducing total cost of RMS multiplication from four down to two exponentiations. This appears challenging because for several reasons: (1) division by s is a nonlinear operation which is challenging to do in HSS, (2) s is also the secret key, which is not known to either party. Our insight is that because $|y| \ll N$ in all existing group-based HSS schemes, each party to locally reduce its share $\langle ys \rangle_\sigma$ modulo N and obtain $\langle y \rangle_\sigma$ with overwhelming probability.

More formally, our scheme ensures that $\langle ys \rangle_\sigma$ is a random subtractive share using shared randomness between the parties (see Section 4.3 and Section 4.4). By definition, $\langle ys \rangle_A = \langle ys \rangle_B + ys$, so $\langle ys \rangle_A \bmod N = (\langle ys \rangle_B \bmod N + y \bmod N) \bmod N$. Since $\langle ys \rangle_B \bmod N$ is uniformly random, then *over the integers*, $\langle y \rangle_A - \langle y \rangle_B = \langle ys \rangle_A \bmod N - \langle ys \rangle_B \bmod N = y$, where the latter equality holds with probability at least $1 - 2^{-\tau}$ when $|y| \leq 2^{-\tau}N$.

2.3.2 Removing Encryptions of x Previously, an input share $\llbracket x \rrbracket_\sigma$ consisted of encryptions of xs and x . It turns out that the ability to derive $\langle x \rangle_\sigma$ directly from $\langle xys \rangle_\sigma$ has an additional benefit: the input share can now be *just* an encryption of xs and not x .

Consider the RMS multiplication operation: we use $\llbracket x \rrbracket_\sigma$ and $\langle y \rangle_\sigma$ to compute the memory share $\langle xy \rangle_\sigma = (\langle xy \rangle_\sigma, \langle xys \rangle_\sigma)$. We describe the process to compute $\langle x'y \rangle_\sigma$, which we can substitute $x' \in \{x, xs\}$ to recover both $\langle xy \rangle_\sigma$ and $\langle xys \rangle_\sigma = \langle (xs)y \rangle_\sigma$:

1. Let $(\text{ct}_0, \text{ct}_1)$ be an encryption of x' .
2. Compute $\langle x'y \rangle_\sigma = \text{DDLog}(\text{ct}_0^{\langle ys \rangle_\sigma} \cdot \text{ct}_1^{\langle y \rangle_\sigma})$.

Note that since we can derive $\langle xy \rangle_\sigma$ *directly* from $\langle xys \rangle_\sigma$ without invoking the procedure above, we only need to invoke the procedure on $x' = xs$ and *not* $x' = x$, removing the need for having additional encryptions of x .

2.3.3 Shortening the Secret Key Recall that the reason Couteau et al. requires the secret key s_σ to be at least as large as N is because in the encryption of x , we require $(N+1)^s \equiv N+1 \in \mathbb{Z}_{N^2}^*$, i.e., $s \equiv 1 \bmod N$. Their major restriction is overcome by our optimization in Section 2.3.2 that removes the need to use encryptions of x . However, it may seem that the constraint introduced in Section 2.3.1 that $y \leq 2^{-\tau}N$ still prevents us from modifying the structure of s_σ . Nevertheless, since we are computing a B -bounded RMS program, we know $y \leq B$, and B is often significantly smaller than N in practice (e.g., $B = 2$ and N being a 3072-bit integer). Let $B' = B \cdot 2^\tau$. Therefore, we can set the secret key to $s_\sigma = t_\sigma \cdot B' + 1$, and later set $\langle y \rangle_\sigma \leftarrow \langle ys \rangle_\sigma \bmod B'$ without breaking correctness. It turns out such secret key structure does not harm security either: see our proof of Theorem 14, specifically Equation (3).

The effect of this optimization is doubled in the joint secret key $s = s_A s_B$. In addition, the length of s no longer depends on the length of N , which allows us to use Paillier-ElGamal ($w = 1$) instead of the generalized DJEG, and work solely over the group $\mathbb{Z}_{N^2}^*$ rather than $\mathbb{Z}_{N^4}^*$.

2.3.4 Reducing Exponent Length Further The primary performance bottleneck of HSS is modular exponentiations, whose running time is linear in the size of the exponent. Every RMS multiplication in this optimized scheme requires computing $\text{ct}_0^{\langle ys \rangle_\sigma} \bmod N^{w+1}$ and $\text{ct}_1^{\langle y \rangle_\sigma} \bmod N^{w+1}$.

Section 2.3.3 automatically reduced $\langle y \rangle_\sigma$ to $B \cdot 2^\tau$ bits. However, $\langle ys \rangle_\sigma$ is still $\log_2(N)$ bits since it is just the output of DDLog. It turns out that a similar local modular reduction trick can be used to reduce the size of $\langle ys \rangle_\sigma$, speeding up the next RMS multiplication.

Specifically, $|xs| \leq |x s_A s_B| \leq B \cdot (B' \cdot 2^{2\lambda})^2 = B^3 \cdot 2^{4\lambda+2\tau}$. To minimize correctness error, we need an extra 2^τ factor, so denote $B'' = B^3 \cdot 2^{4\lambda+3\tau}$. We can then set $\langle ys \rangle_\sigma \leftarrow \langle ys \rangle_\sigma \bmod B''$ and retain correctness with overwhelming probability. This is similar to an optimization in [DZX⁺24, Section V.B] where they *transform* the output of DDLog to a smaller modulus. We remark that for practical parameter sizes used for ANIKE, $\log_2(B'') \approx \log_2(N)/3$, so this roughly provides a $3\times$ speedup.

2.3.5 Reducing Input Length to NIM Recall that in order to obtain the initial memory share $\langle 1 \rangle_\sigma$, each party needs to obtain a subtractive share of the joint secret key $s = s_A s_B$. Setting aside the details, a non-interactive multiplication (NIM) procedure allows Alice and Bob, holding inputs x, y respectively, to obtain a secret share of xy . Couteau et al. [CDH⁺25] obtains $\langle s \rangle_\sigma$ using the naïve way: by having Alice set $x = s_A$ and Bob set $y = s_B$.

Internally, NIM is implemented with expensive modular exponentiations, so it helps to shorten the NIM inputs x, y as much as possible. Recall our secret key is of a special structure $s_\sigma = t_\sigma \cdot B' + 1$, so it is natural to ask whether we can set $x = t_A$ and $y = t_B$ as NIM inputs, obtain $\langle t_A t_B \rangle_\sigma$, and do some manipulations to transform it into $\langle s \rangle_\sigma$.

To achieve this, we observe that $s = s_A s_B = (t_A B' + 1)(t_B B' + 1) = t_A t_B (B')^2 + (t_A + t_B) B' + 1$, and that $\langle t_A + t_B \rangle_A = t_A$, $\langle t_A + t_B \rangle_B = -t_B$ is a valid subtractive share. Finally, the result $\langle s \rangle_\sigma$ can be obtained as a linear combination: $\langle s \rangle_\sigma = \langle t_A t_B \rangle_\sigma (B')^2 + \langle t_A + t_B \rangle_\sigma B' + \langle 1 \rangle_\sigma$.

2.4 Correctness

We now pay close attention to the condition which ensures the correctness of our optimized scheme. Let l_N denote the bit-length of the modulus N , and w be the DJEG parameter, such that the ciphertext modulus is N^{w+1} . In order to ensure the $\langle ys \rangle_\sigma$ parts of a memory share of y remain subtractive shares over $\mathbb{Z}$ after each DDLog procedure, we need $|ys| \leq N^w$ even when we *do not* artificially reduce modulo B'' in Section 2.3.4. Therefore, we need $N^w \geq B'' = B^3 \cdot 2^{4\lambda+3\tau}$. In other words, $w l_N \geq 3 \log_2(B) + 4\lambda + 3\tau$.

To deal with arbitrarily large message bounds B , we may increase either l_N or w , whichever is more efficient. We discuss in Section 8.1.3 how we optimally increase those parameters to tolerate large bounds B for evaluation.

We refer readers who are interested in seeing the full definition of MKHSS, details regarding our construction, and our proof of correctness and security to Section 6.

3 Technical Overview: ANIKE

Attribute-based non-interactive key exchange (ANIKE) is a primitive that solves the following problem: two parties, each holding private inputs x_A and x_B , want to agree on a common key if and only if their inputs satisfy some policy P : $P(x_A, x_B) = 0$, without revealing to each other any information beyond whether P is satisfied. This primitive naturally lends itself to applications to location-based and fuzzy password-authenticated key exchange (PAKE), by observing that P can be a predicate that determines whether x_A and x_B are *close* in some metric.

Moreover, ANIKE is *non-interactive* in the same sense as a non-interactive key exchange (NIKE) [FHKP13] (e.g., Diffie-Hellman [DH76]). The key exchange protocol works by having each party run an algorithm `AttrKeyDer` to create a public encoding pe_σ of its private attribute x_σ , exchange the public encodings, and the parties locally deriving a key via `AttrKeyGen`. The protocol is illustrated in Figure 1.

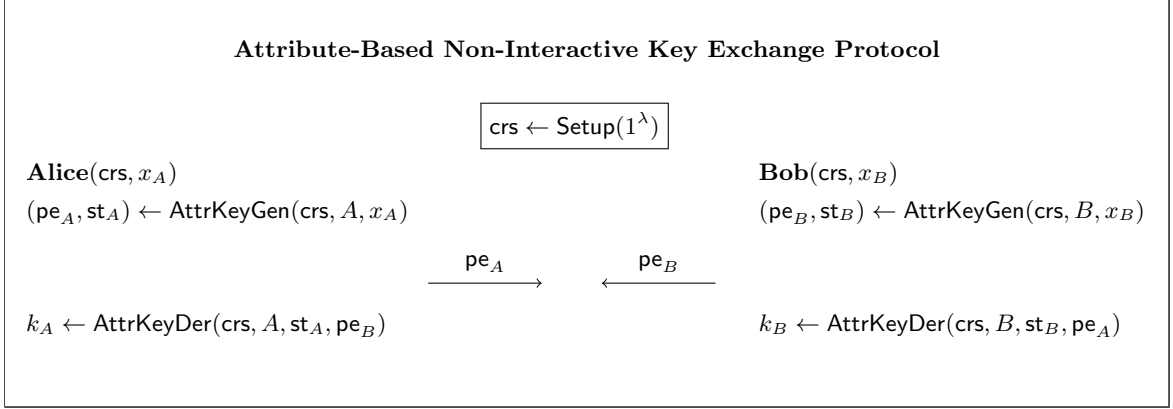


Fig. 1: The ANIKE protocol. See Definition 13 for the formal syntax of the algorithms AttrKeyGen and AttrKeyDer .

Couteau et al. [CDH⁺25] showed the theoretical feasibility of ANIKE from MKHSS that supports the same class of predicates as MKHSS, i.e., RMS programs, or logarithmic-space computation.

3.1 Our Contributions

We show that ANIKE can be concretely realized with similar efficiency as our MKHSS scheme on the same predicate. In addition, we show that our ANIKE can realize predicates with real-world applications to a geolocation-based key exchange and fuzzy PAKE.

3.1.1 Application: Geolocation-Based Key Exchange Consider the case where each party's input is a latitude-longitude pair $x_\sigma = (u_\sigma, v_\sigma)$, and the predicate $P(x_A, x_B)$ is satisfied when x_A and x_B represent geographically close locations. A canonical application is to use location services (e.g., GPS) to authenticate a key exchange between geographically close devices, where the devices need not be physically close, e.g., on different floors in a skyscraper. We also present another application that uses self-reported location information instead of location services. Consider a traveler who wishes to contact local residents near his travel destination to discuss arrangements for a homestay (à la AirBnB). The traveler may wish to only consider stays within a certain distance d from his destination (e.g., a conference venue), so we only want parties that satisfy this location proximity to successfully complete the key exchange. The resident wants to hide her home address (until she agrees to host a traveler) to ensure privacy, and the traveler may similarly not wish to disclose the exact destination. The non-interactivity of ANIKE allows each party to engage in this protocol asynchronously without being simultaneously online.

3.1.2 Application: Fuzzy PAKE Password-authenticated key exchange (PAKE) [BPR00] allows two parties to agree on a common key using a shared password. It is useful when a user wants to configure two personal devices to agree on the same key. Traditional password-based protocols, such as that of Facebook [Pro15], often allow authentication under a small set of common typos to enhance usability. However, it is challenging to equip PAKE with a fuzzy notion of equality. Unlike plain PAKE, prior constructions of fuzzy PAKE [DHP⁺18] for rich notions of fuzziness are based on MPC and require several rounds of interaction. See Section 1.1 for the importance of non-interactivity.

We focus on the case where the password is a passphrase. It is well known that passphrases provide a good balance between memorability and security.⁶ We model a passphrase as a list of relatively short strings, whose contents can be arbitrary characters from an exponentially large set (e.g., lowercase letters, ASCII, UTF-16, etc.). In this model, we are able to successfully complete a key exchange despite a limited number of substitutions, transpositions, insertions, and deletions, and do so for realistic passphrase and word lengths.

⁶ This is captured by this famous XKCD comic: <https://xkcd.com/936/>.

Importance of fuzziness. A study [CAA⁺16] found that the lack of fuzzy matching significantly increase login delay (and prevented some users from *ever* completing a successful login) *even* in the setting where a user is given nearly instant feedback when an incorrect password is entered. Note that the non-interactive nature of the fuzzy PAKE protocol *delays* such feedback in most scenarios. For example, in the scenario of key agreement between personal devices, the user only knows if the key exchange succeeds *after* both personal devices use the key in another protocol.

3.1.3 Multi-Factor Authenticated Key Exchange We show it is possible to combine arbitrary predicates via logical conjunctions and disjunctions. This enables a form of *multi-factor* authenticated key exchange, e.g., keys agree if the location is close *and* the password match. For example, a system administrator may wish to configure two corporate-owned devices to agree on the same key using a remembered password, where the devices are located in different rooms in the same building. In this scenario, using a combined location proximity and password will provide better security.

There are other interesting compositions. For the homestay example, the traveler may have additional requirements such as an acceptable price range, and such a numerical interval comparison can be seen as a simpler 1-dimensional case of the location proximity predicate.

We stress that since this composition works at the *predicate* level, it requires no extra rounds of communication or protocol modifications: the non-interactivity and reusability property of ANIKE is preserved. The cost (public encoding size and computation time) is at most a sum over the cost of running the protocol over individual predicates.

3.2 Starting Point: Couteau et al.

Couteau et al. [CDH⁺25] provides a theoretical solution to ANIKE via black-box use of an MKHSS scheme with external security, which our MKHSS scheme achieves. External security requires that the subtractive shares of the output look pseudorandom conditioned on the value of the output. We describe their idea below:

A natural idea to transform MKHSS into key exchange is to observe that subtractive shares $\langle x \rangle = (\langle x \rangle_A, \langle x \rangle_B)$ have the following nice property: $\langle x \rangle_A = \langle x \rangle_B$ if and only if $x = 0$. Since the output of MKHSS is a subtractive share, an extremely naïve solution is to make MKHSS compute the policy function $P(x, y)$ directly and let $k_\sigma = \langle P(x, y) \rangle_\sigma$.

One problem is that even when $P(x, y) \neq 0$, the shares are still correlated. For example, if Alice knows that $P(x, y)$ is a small integer, she can easily brute force and obtain Bob’s key. An immediate fix is to have each party encode a secret shift value $\Delta_\sigma \in [2^\lambda]$ into an additional input share, and have MKHSS compute $P(x, y) \cdot \Delta_A \cdot \Delta_B$ instead.

Recall that reusability is a crucial requirement of ANIKE. Unfortunately, the encoding of the secret shift is not reusable. Consider a client-server authentication protocol that uses ANIKE as a building block, where after two parties obtain keys k_A, k_B , one party (say Alice, the client) sends over her key k_A to the server Bob, who then checks whether $k_A = k_B$. If Bob is the adversary, he can calculate $k_B - k_A = \Delta_A \cdot \Delta_B \cdot P(x, y)$. Suppose Alice failed the authentication procedure, and $P(x, y) = 1$, then Bob could recover Alice’s long-term secret shift $\Delta_A = (k_B - k_A)/\Delta_B$. This compromises the security of future key exchanges.

To achieve reusability, instead of encoding Δ_σ directly in an input share, Couteau et al. makes each party encode a PRF key K_σ in its long-term public encoding. Then, for each $\sigma \in \{A, B\}$, compute $\Delta_\sigma = F(K_\sigma, \text{id}_A \parallel \text{id}_B \parallel P)$ inside the RMS program. This way, for every key exchange session, the PRF generates an independently random shift.

3.3 Making Generic ANIKE Efficient

The major inefficiency in their construction is the need to evaluate a pseudorandom function (PRF) *homomorphically* over values inside the RMS program, making it impractical to implement, *even* with an efficient MKHSS scheme.

Instead, we observe that only a small modification is needed to secure the otherwise insecure approach where each party σ encodes Δ_σ . The idea is to let the key be a *hash* of the subtractive share along with a unique value for each session: $k_\sigma = H(\langle P(x, y) \cdot \Delta_A \cdot \Delta_B \rangle_\sigma \parallel \text{id}_A \parallel \text{id}_B \parallel P)$. We model the hash function as a random oracle, and security follows from the correlation-robustness of hashing.

Construction 1: RMS program to compute whether $a < b$.

The L -bit integers x, y are encoded in synchronized input shares that encode their binary decompositions: $\{\{x\}\} = (\{x_1\}, \dots, \{x_L\})$ and $\{\{y\}\} = (\{y_1\}, \dots, \{y_L\})$. $\langle\langle a \rangle\rangle_\sigma$ denotes a memory share of a . We denote $\bar{a} := 1 - a$.

LessThan($\{\{x\}\}, \{\{y\}\}, \langle\langle z_0 \rangle\rangle_\sigma$)

```

1 : for  $i$  in  $\{1, \dots, L\}$ ,
2 :    $\langle\langle x_i z_{i-1} \rangle\rangle_\sigma \leftarrow \{x_i\} \cdot \langle\langle z_{i-1} \rangle\rangle_\sigma$ 
3 :    $\langle\langle \bar{x}_i z_{i-1} \rangle\rangle_\sigma \leftarrow \langle\langle z_{i-1} \rangle\rangle_\sigma - \langle\langle x_i z_{i-1} \rangle\rangle_\sigma$ 
4 :    $\langle\langle \bar{x}_i y_i z_{i-1} \rangle\rangle_\sigma \leftarrow \{y_i\} \cdot \langle\langle \bar{x}_i z_{i-1} \rangle\rangle_\sigma$ 
5 :    $\langle\langle \bar{x}_i \bar{y}_i z_{i-1} \rangle\rangle_\sigma \leftarrow \langle\langle \bar{x}_i z_{i-1} \rangle\rangle_\sigma - \langle\langle \bar{x}_i y_i z_{i-1} \rangle\rangle_\sigma$ 
6 :    $\langle\langle x_i y_i z_{i-1} \rangle\rangle_\sigma \leftarrow \{y_i\} \cdot \langle\langle x_i z_{i-1} \rangle\rangle_\sigma$ 
7 :    $\langle\langle z_i \rangle\rangle_\sigma \leftarrow \langle\langle \bar{x}_i \bar{y}_i z_{i-1} \rangle\rangle_\sigma + \langle\langle x_i y_i z_{i-1} \rangle\rangle_\sigma$ 

8 :  $\langle\langle z_0 \cdot \mathbb{1}(x < y) \rangle\rangle_\sigma \leftarrow \sum_{i=1}^L \langle\langle \bar{x}_i y_i z_{i-1} \rangle\rangle_\sigma$ 

9 : return  $\langle\langle z_0 \cdot \mathbb{1}(x < y) \rangle\rangle_\sigma$ 

```

However, when we instantiate our MKHSS scheme for this predicate, we require increasing the bound to $B \geq 2\lambda$ so that correctness (and security) hold(s). Fortunately, due to the fact that our memory share of $P(x, y)$ is already of the format $\langle\langle P(x, y) \rangle\rangle_\sigma = (\langle\langle P(x, y) \cdot s \rangle\rangle_\sigma, \langle\langle P(x, y) \rangle\rangle_\sigma)$, we can hash the former element, and use s in place of $\Delta_\sigma \cdot \Delta_{1-\sigma}$. It provides the same security guarantee, since the joint secret $s = s_A s_B$ is unknown to any single party, again relying on modeling the hash as a random oracle.

Lastly, we replace $\text{id}_A \parallel \text{id}_B$ with a key k_{nike} obtained through any non-interactive key exchange (e.g., Diffie-Hellman) that runs in parallel to our protocol. This serves not only as a replacement of the identity strings, but crucially ensures that the resulting keys look pseudorandom to external parties even when the predicate is satisfied, without extra requirements on the output of the MKHSS scheme. In the end, the key derivation procedure computes $k_\sigma = H(\langle\langle P(x, y) \cdot s \rangle\rangle_\sigma \parallel k_{\text{nike}} \parallel P)$.

3.4 Geolocation-Based Key Exchange

After showing a generic ANIKE construction, it remains to show how to use it with practically interesting policies from efficiently computable RMS programs. The first predicate we consider is location proximity. We model a party's location $\mathbf{x}_\sigma$ as a D -dimensional vector of L -bit integers, and the predicate $P(\mathbf{x}_A, \mathbf{x}_B) = 0$ if and only if $\mathbf{x}_A$ and $\mathbf{x}_B$ are within a distance d according to some metric.

For simplicity of presentation, we focus on a simple example where the location of each party can be described as a two-dimensional coordinate in an Euclidean plane, i.e., each party's attribute is a vector of two integers: $\mathbf{x}_\sigma = (u_\sigma, v_\sigma)$. We define closeness via the Chebyshev distance, i.e., $\text{CD}(\mathbf{x}_A, \mathbf{x}_B) = \max\{|u_A - u_B|, |v_A - v_B|\}$. In other words, two parties agree on a key if $\mathbf{x}_B$ is located within a square area centered around $\mathbf{x}_A$. Observe that $\text{CD}(\mathbf{x}_A, \mathbf{x}_B) < d \iff |u_A - u_B| < d \wedge |v_A - v_B| < d$. In fact, we can decompose a single comparison further: $|u_A - u_B| < d \iff u_A > u_B - d \wedge u_A < u_B + d$.

By letting each party $\sigma \in \{A, B\}$ provide shares that encode integers $u_\sigma, u_\sigma - d, u_\sigma + d, v_\sigma, v_\sigma - d, v_\sigma + d$, this reduces to performing comparison $x < y$ using an RMS program. In fact, we further reduce the number of shares by letting the parties (denote Alice and Bob) play asymmetric roles. Alice just needs to share u_A and v_A , and Bob just needs to share $u_\sigma \pm d$ and $v_\sigma \pm d$.

Lastly, we show how to do the integer comparison $x < y$. We let the input shares encode a binary representation of x and y , so integer comparison can be seen as a branching program of size linear in the bit lengths of x and y using the simple algorithm of bit-by-bit comparison, and we provide an explicit RMS program in Construction 1.

To see correctness, observe that in line 7, `LessThan` recursively defines $z_i = (\bar{x}_i \bar{y}_i + x_i y_i) z_{i-1} = z_{i-1} \cdot \mathbb{1}(x_i = y_i)$. Thus, $z_i = z_0 \cdot \mathbb{1}(x_1 \dots x_i = y_1 \dots y_i)$, i.e., if $z_0 = 1$ and the prefix of length i matches.

Observe that $x < y$ if and only if the binary decompositions differ, and in the first bit position i where they differ, $x_i = 0$ and $y_i = 1$. Using logic notation, this can be described as $\mathbb{1}(x < y) = \bigvee_{i=1}^L (\bar{x}_i \wedge y_i \wedge \mathbb{1}(x_1 \dots x_{i-1} = y_1 \dots y_{i-1}))$, where it is easy to see that the clauses across different indices i are mutually exclusive. Therefore, line 8 can be seen as computing this big disjunction via a summation.

If the goal is to just compute $x < y$, we can set $\langle\langle z_0 \rangle\rangle_\sigma = \langle\langle 1 \rangle\rangle_\sigma$, but we made the algorithm take in $\langle\langle z_0 \rangle\rangle_\sigma$ as an argument to allow computing logical conjunctions over multiple comparisons (recall it is not possible to multiply memory values in an RMS program). For example, to compute $x < y \wedge v < w$, we can do:

1. $\langle\langle \mathbb{1}(x < y) \rangle\rangle_\sigma \leftarrow \text{LessThan}(\{\{x\}\}, \{\{y\}\}, \langle\langle 1 \rangle\rangle_\sigma)$
2. **return** $\text{LessThan}(\{\{v\}\}, \{\{w\}\}, \langle\langle \mathbb{1}(x < y) \rangle\rangle_\sigma)$

which will be needed for our location proximity ANIKE predicate. This will also enable *composing* the location predicate with other extra requirements. Using the homestay example, it could be price ranges, amenities, etc. In addition, using more conjunctions allows this predicate to be generalized to higher dimensions $D \geq 3$, by having Alice provide shares $x_A^{(1)}, \dots, x_A^{(D)}$ (instead of u_A, v_A) and Bob provide shares $x_B^{(1)} \pm d, \dots, x_B^{(D)} \pm d$ (instead of $u_B \pm d, v_B \pm d$).

Using a similar technique, multiple location predicates can also be composed using logical disjunctions, e.g., allowing key exchange between devices located in any one of the corporate headquarters.

3.4.1 Generalizations We presented a predicate that checks whether the Chebyshev distance is under a threshold d in Euclidean space, i.e., whether $\mathbf{x}_B$ is within a hypercube region around $\mathbf{x}_A$. We define this square region as the *accepting region* of this predicate. We remark that our scheme is very versatile and supports many extensions that alter this *accepting region* to support practical scenarios, like geographical coordinates.

Hyper-rectangular accepting regions. We can easily make this accepting region an arbitrary hyperrectangle by having different thresholds $d_1, \dots, d_D$ for dimensions 1 through D .

Then, observe that for a key exchange to succeed, we only need the shares $x_A^{(1)}, \dots, x_A^{(D)}$ from one party (say Alice), and $x_B^{(1)} \pm d_1, \dots, x_B^{(D)} \pm d_D$ from Bob. Therefore, instead of setting $d_1, \dots, d_D$ as system parameters, we can allow one party (here Bob) to pick them. This is already useful in the homestay example, where the traveler may want to decide the distance depending on commute time preferences.

Geographical coordinates. Geolocation coordinates are spherical rather than Euclidean coordinates. Therefore, to compare whether two latitude-longitude pairs are close (under some relatively small metric distance threshold m), we need to account for the conversion factor α such that a metric difference of 1 meter corresponds to an angular difference of α degrees in latitude and β degrees in longitude, i.e., set $d_1 = \alpha m$ and $d_2 = \beta m$. While α is fixed, β depends on the approximate latitude of both parties, so when m is small (implying both parties must lie on similar latitude for authentication to succeed), we need one party to provide d_2 , which our scheme supports.

An alternative approach is to convert the spherical coordinates to a *three-dimensional* Euclidean coordinate describing the displacement from Earth's center, if one is willing to tolerate the additional cost of approximating Euclidean distance with Chebyshev distance in three dimensions. The maximum deviation between Chebyshev distance and Euclidean distance is $\sqrt{D}$, which in $D = 3$ dimensions, is 73%.

Approximate Euclidean distance. Observe, we can make the accepting region an intersection of many rotated hypercubes, by having each party provide shares of their coordinates in rotated coordinate systems, and making the predicate a logical conjunction. It is easy to see that in two dimensions, the intersection of sufficiently many rotated squares approximate a circle, so this can be used to arbitrarily approximate a Euclidean distance threshold in low dimensions, providing a more precise matching.

Construction 2: RMS program to compute whether the Hamming distance between two strings $x, y \in \{0, 1\}^L$ is strictly greater than a threshold T .

The strings $x, y \in \{0, 1\}^L$ are represented by vectors of input shares $\{\mathbf{x}\} = (\{x_1\}, \dots, \{x_L\})$ and $\{\mathbf{y}\} = (\{y_1\}, \dots, \{y_L\})$. $\langle\langle v \rangle\rangle_\sigma$ denotes a memory share of v .

$\text{ExceedsThreshold}_L(\{\mathbf{x}\}, \{\mathbf{y}\}, T, \langle\langle z_0 \rangle\rangle_\sigma)$

```

1: for  $j$  in  $\{0, \dots, T + 1\}$ ,
2:    $\langle\langle a_{0,j} \rangle\rangle_\sigma \leftarrow \langle\langle z_0 \rangle\rangle_\sigma$  if  $j = 0$  else  $\langle\langle 0 \rangle\rangle_\sigma$ 
3: for  $i$  in  $\{1, \dots, L\}$ ,
4:    $\langle\langle za_{i-1,(-1)} \rangle\rangle_\sigma := \langle\langle 0 \rangle\rangle_\sigma$ 
5:   for  $j$  in  $\{0, \dots, T\}$ ,
6:      $\langle\langle za_{i-1,j} \rangle\rangle_\sigma \leftarrow \text{Differ}(\{x_i\}, \{y_i\}, \langle\langle a_{i-1,j} \rangle\rangle_\sigma)$ 
7:      $\langle\langle \bar{z}a_{i-1,j} \rangle\rangle_\sigma \leftarrow \langle\langle a_{i-1,j} \rangle\rangle_\sigma - \langle\langle za_{i-1,j} \rangle\rangle_\sigma$ 
8:      $\langle\langle a_{i,j} \rangle\rangle_\sigma \leftarrow \langle\langle za_{i-1,j-1} \rangle\rangle_\sigma + \langle\langle \bar{z}a_{i-1,j} \rangle\rangle_\sigma$ 
9:  $\langle\langle z_0 P(x, y) \rangle\rangle_\sigma \leftarrow \sum_{i=0}^{L-1} \langle\langle za_{i,T} \rangle\rangle_\sigma$ 
10: return  $\langle\langle z_0 P(x, y) \rangle\rangle_\sigma$ 

```

$\text{Differ}(\{x\}, \{y\}, \langle\langle z_0 \rangle\rangle_\sigma)$

```

1:  $\{x - y\} \leftarrow \{x\} - \{y\}$ 
2:  $\langle\langle (x - y)z_0 \rangle\rangle_\sigma \leftarrow \{x - y\} \cdot \langle\langle z_0 \rangle\rangle_\sigma$ 
3:  $\langle\langle (x - y)^2 z_0 \rangle\rangle_\sigma \leftarrow \{x - y\} \cdot \langle\langle (x - y)z_0 \rangle\rangle_\sigma$ 
4: return  $\langle\langle (x - y)^2 z_0 \rangle\rangle_\sigma$ 

```

3.5 Fuzzy PAKE

3.5.1 Formalization We define a passphrase as a sequence of W words of length at most L , whose characters come from an alphabet Σ . Formally, a passphrase is a vector of words $\mathbf{x} = x_1, \dots, x_W \in (\Sigma \sqcup \{\perp\})^L$, where each word x_i is a string potentially padded with $\perp$ symbols until its length is L .

We describe the predicate $P(\mathbf{x}, \mathbf{y})$ we use to capture real-world mistakes, which is parameterized by two thresholds Q and T :

1. For each index i , we permit x_i and y_i to differ by at most Q single character substitutions without penalty. This accounts for substitution and (to a lesser extent) transposition errors.
2. Let E be the number of corresponding words that differ by more than Q substitutions. We define $P(\mathbf{x}, \mathbf{y}) = 1$ if and only if $E \leq T$. This way, we can tolerate up to T insertions, deletions, and even forgetting entire words.

3.5.2 Warmup: Hamming Distance We first show how to construct an RMS program for a simple predicate where $P(\mathbf{x}, \mathbf{y}) = 1$ if the Hamming distance between two binary strings (treated as a vector of bits) $\mathbf{x}, \mathbf{y} \in \{0, 1\}^L$ is strictly greater than T , and $P(\mathbf{x}, \mathbf{y}) = 0$ otherwise.

First, we observe that the natural way to compute this using pseudocode is to declare a zero-initialized accumulator that stores the number of differing bits, iterate through the string, and in each iteration, increment the accumulator conditioned on the bits differing.

The idea is to store the accumulator as a one-hot encoding in L memory values. Then in each iteration, update the accumulator based on the input bit values. We present a detailed algorithm in Construction 2. To do single-level threshold matching, we invoke $\text{ExceedsThreshold}_L(\{\mathbf{x}\}, \{\mathbf{y}\}, T, \langle\langle 1 \rangle\rangle_\sigma)$.

To verify correctness, observe the invariant that $\mathbf{a}_i = (a_{i,0}, \dots, a_{i,T})$ is a one-hot encoding of the number of differing bits in the first i bits of x and y if the value is at most T , and zero otherwise. In addition, $za_{i,T}$ can be viewed an *overflow flag* for the accumulator $\mathbf{a}_i$ that indicates if an overflow occurred exactly after reaching the i -th bit, so summing over this value for all i determines whether the threshold T was ever exceeded.

3.5.3 Supporting Multiple Thresholds To support more than one level of threshold, we observe that we can replace `Differ` with recursive calls to `ExceedsThreshold` inside itself.

We focus on a concrete example that is almost our two-level passphrase threshold (i.e. L words each of W characters), except the alphabet is still binary: $\Sigma \in \{0, 1\}$: where we allow Q incorrect characters (bits in this simplified example) and T incorrect words.

We first define `ExceedsThresholdW` to be essentially the definition in Construction 2, except with W in place of L . We then redefine `ExceedsThresholdL` to instead take in an $L \times W$ matrix of input values $\{\{\mathbf{X}\}\}$ and $\{\{\mathbf{Y}\}\}$, where $X_{i,j}$ is the j -th character of the i -th word. We then replace line 6 of `ExceedsThresholdL` with a recursive call to `ExceedsThresholdW` as follows

$$\langle\langle z \rangle\rangle_\sigma \leftarrow \text{ExceedsThreshold}_W(\{\{\mathbf{x}_i\}\}, \{\{\mathbf{y}_i\}\}, \langle\langle a_{i-1,j} \rangle\rangle_\sigma),$$

where $\mathbf{x}_i$ and $\mathbf{y}_i$ are the i -th rows of the matrices $\mathbf{X}$ and $\mathbf{Y}$, respectively.

Lastly, we observe that to extend to three or more levels, we can make $\mathbf{X}$ and $\mathbf{Y}$ tensors, and recurse for more levels.

3.5.4 Supporting Arbitrary Alphabets To extend the above program to an exponentially-large alphabet Σ , we first encode each character as a binary string of b bits. It suffices to have $b = \log_2(|\Sigma| + 1)$, i.e., every character in Σ and the special $\perp$ symbol can be encoded. We then treat character comparison as itself a trivial threshold matching problem with the threshold being zero (a single bit of difference makes two characters unequal). This means that our fuzzy PAKE example can be thought of as having three levels of threshold (0, Q , and T), and the input is a three-dimensional tensor of size $L \times W \times b$.

3.5.5 Optimizations If we put together the accumulator values $a_{i,j}$ as a matrix $\mathbf{A}$ (i.e., stacking together one-hot encodings $\mathbf{a}_i$ as column vectors), then since the accumulator value starts at zero and increases by at most one in every iteration, it is easy to see that the lower left triangle of the matrix always consists of all zeroes. In addition, the upper right triangle of the matrix does not contribute to the output. Therefore, Line 6 in Construction 2 need not be executed for every combination of i, j . This saves calls to `Differ`, in addition to *recursive* calls to `ExceedsThreshold`, which provides significant savings in the number of RMS multiplications.

4 Preliminaries

We denote $\text{negl}(\cdot)$ a negligible function: a function that is asymptotically smaller than any inverse polynomial. For two random distributions $\mathcal{D}_0, \mathcal{D}_1$, denote $\mathcal{D}_0 \approx_p \mathcal{D}_1$, $\mathcal{D}_0 \approx_s \mathcal{D}_1$, and $\mathcal{D}_0 \approx_c \mathcal{D}_1$ to mean that the distributions are perfectly, statistically, and computationally indistinguishable, respectively. We use either $x \equiv y \pmod{M}$ or $x \equiv_M y$ to denote that the integers x and y are equal modulo M .

4.1 Assumptions

We list the assumptions we use when proving computational security properties:

Definition 1 (Decisional Composite Residuosity (DCR) Assumption [Pai99]). Let `GenPQ` be a randomized algorithm that, on input the security parameter λ , outputs two distinct, sufficiently large, random safe primes p and q . The DCR assumption states that:

$$\left\{ (N, g_0) \mid \begin{array}{l} (p, q) \leftarrow \text{GenPQ}(1^\lambda) \\ N := pq \\ g_0 \leftarrow \$_{\mathbb{Z}_{N^2}^*} \end{array} \right\} \approx_c \left\{ (N, g_1) \mid \begin{array}{l} (p, q) \leftarrow \text{GenPQ}(1^\lambda) \\ N := pq \\ g_0 \leftarrow \$_{\mathbb{Z}_{N^2}^*} \\ g_1 := g_0^N \end{array} \right\}.$$

Definition 2 (G-Short Exponent Indistinguishability (SEI) Assumption; Adapted from [ABC⁺24]). Let $\mathbb{G}$ be a group, and $g \in \mathbb{G}$ be a generator of the group. Let λ be a security parameter. The $\mathbb{G}$ -SEI assumption states that:

$$\left\{ (g, g^r) \mid r \leftarrow \$_{[2^{2\lambda}]} \right\} \approx_c \left\{ (g, g') \mid g' \leftarrow \$_{\mathbb{G}} \right\}.$$

Remark 3. [ABC⁺24] shows that SEI is equivalent to the seemingly weaker short exponent discrete log (SEDL) assumption. SEDL is a computational version of the decisional problem of SEI: SEDL says it is hard to actually compute the short exponent r when given g^r , rather than simply distinguishing between g^r and a random group element like in SEI.

Remark 4. Throughout our construction, we mostly focus on the subgroup $\mathbb{G}$ of $2N^w$ -th residues in $\mathbb{Z}_{N^{w+1}}^*$ for some integer w , in which discrete logarithm is hard assuming DCR. In this group, one cannot perfectly sample a random element $g' \leftarrow \$ \mathbb{G}$ without knowing the group order $\phi(N)/4$, which would allow factoring N . However, we can obtain a statistically close sample by sampling $r' \leftarrow \$ [N]$ or $r' \leftarrow \$ [N^w]$ and letting $g' = g^{r'}$, so this assumption is equivalent to the right hand side being sampled from those distributions, and we exploit this fact in our constructions.

4.2 D  mg  rd-Jurik-ElGamal Encryption

Construction 3: A variant of the Damg  rd-Jurik-ElGamal encryption scheme.

Parameters. The exponent parameter $w \geq 1$ determines the plaintext space, which are integers with absolute value bounded by $N^w \cdot 2^{-2\lambda-1}$. S_{sk} is a distribution where the secret key is sampled, subject to Equation (1).

Definitions. Algorithm **GenPQ** is defined in Definition 1. Let L_w be a function that efficiently computes the discrete logarithm base $N + 1$ in the group $\mathbb{Z}_{N^w}^*$ [DJ01].

DJEG.Setup(1^λ): 1 : $(p, q) \leftarrow \text{GenPQ}(1^\lambda)$ 2 : $N := pq$ 3 : $g_0 \leftarrow \$ \mathbb{Z}_{N^{w+1}}^*$ 4 : $g := (g_0)^{2N^w} \in \mathbb{Z}_{N^{w+1}}^*$ 5 : return $\text{crs} := (N, g)$	DJEG.KeyGen(crs): 1 : parse $\text{crs} = (N, g)$ 2 : $s \leftarrow \$ S_{\text{sk}}$ 3 : $f := g^{-s} \in \mathbb{Z}_{N^{w+1}}^*$ 4 : $(\text{pk}, \text{sk}) := (f, s)$ 5 : return (pk, sk)
--	---

DJEG.Encrypt(crs, pk, x): 1 : parse $\text{crs} = (N, g)$ 2 : parse $\text{pk} = f$ 3 : $r \leftarrow \$ [2^{2\lambda}]$ 4 : $c_0 := g^r \bmod N^{w+1}$ 5 : $c_1 := (N + 1)^x f^r \bmod N^{w+1}$ 6 : return $\text{ct} := (c_0, c_1)$	DJEG.FlipEncrypt(crs, pk, x): 1 : parse $\text{crs} = (N, g)$ 2 : parse $\text{pk} = f$ 3 : $r \leftarrow \$ [2^{2\lambda}]$ 4 : $c_0 := (N + 1)^x g^r \bmod N^{w+1}$ 5 : $c_1 := f^r \bmod N^{w+1}$ 6 : return $\text{ct} := (c_0, c_1)$	DJEG.Decrypt($\text{crs}, \text{sk}, \text{ct}$): 1 : parse $\text{crs} = (N, -)$ 2 : parse $\text{ct} = (c_0, c_1)$ 3 : parse $\text{sk} = s$ 4 : $c' := c_1 \cdot (c_0)^s$ 5 : $x := L_w(c')$ 6 : return x
--	--	--

Damg  rd-Jurik-ElGamal (DJEG) cryptosystem [DJ03,CDH⁺25] is a natural generalization of Paillier-ElGamal [BCP03] cryptosystem. In Construction 3, we provide a variant of DJEG that is similar to the construction used by Couteau et al. [CDH⁺25]. There are two major differences: (1) our variant uses short exponents (Definition 2) for the encryption randomness, and (2) to support our MKHSS construction, we generalize the scheme to support any efficiently sampleable secret key distribution S_{sk} subject to the following computational indistinguishability condition:

$$\{(g, g^s) : s \leftarrow \$ S_{\text{sk}}\} \approx_c \{(g, g') : g' \leftarrow \$ \mathbb{G}\}, \quad (1)$$

where g is a generator of $\mathbb{G}$, the subgroup of $2N^w$ -th residues of $\mathbb{Z}_{N^{w+1}}^*$.

For example, $S_{\text{sk}} = [2^{2\lambda}]$ will satisfy the above assuming SEI. We use a more structured secret key distribution in our MKHSS construction.

We briefly describe some important properties of DJEG not typically found in conventional encryption schemes, that turns out to be very useful for MKHSS:

1. The key generation is split into two phases, **Setup** and **KeyGen**. **Setup** only needs to be run once by a trusted party, and the *same* common reference string (CRS) can be reused by multiple parties that run **KeyGen** to generate independent keys.
2. There is an additional **FlipEncrypt** algorithm to generate a (circular) encryption of xs , where s is the *secret* key, with the same semantic security guarantees as **Encrypt**. Since the algorithm does not require the secret key as an input, it means that we do not need to make extra key-dependent message (KDM) security assumptions.

We can verify the correctness of **Encrypt**. Let c' be the variable of the same name inside **Decrypt**, we have:

$$\begin{aligned} c' &= ((N+1)^x f^r)(g^r)^s \\ &= (N+1)^x (g^{-s})^r (g^r)^s \\ &= (N+1)^x, \end{aligned}$$

then $L_w(c') = x \bmod N^w$, and since we know $|x| \leq N^w \cdot 2^{-2\lambda-1}$, we can recover x by subtracting a multiple of N^w if necessary (to handle negative numbers).

The correctness of **FlipEncrypt** can be verified similarly:

$$\begin{aligned} c' &= (f^r)((N+1)^x g^r)^s \\ &= (g^{-s})^r (N+1)^{xs} (g^r)^s \\ &= (N+1)^{xs}, \end{aligned}$$

then $L_w(c') = xs \bmod N^w$, and since we know $|xs| \leq N^w/2$, we can recover xs by subtracting a multiple of N^w if necessary.

It is also easy to verify that the scheme is linearly homomorphic. We then show security:

Lemma 5. *Let λ be a security parameter. Let $\mathbb{G}$ is the subgroup of $2N^w$ -th residues over $\mathbb{Z}_{N^{w+1}}^*$. If the DCR assumption and $\mathbb{G}$ -SEI holds, then the encryption scheme presented in Construction 3, specifically **Encrypt** and **FlipEncrypt**, satisfies the standard notion of semantic security (i.e., CPA-security).*

Proof. The proof is very similar to the security proof for DJEG in Couteau et al. [CDH⁺25, Lemma 5]. The major differences are in hybrid $\mathcal{H}_1$ and $\mathcal{H}_2$, where we account for the differences in the distribution of encryption randomness and that of the secret key, respectively.

- *Hybrid $\mathcal{H}_0$.* This hybrid consist of a ciphertext (c_0, c_1) as generated by DJEG.Encrypt in Construction 3.
- *Hybrid $\mathcal{H}_1$.* In this hybrid game, we sample the encryption randomness as $r \leftarrow_{\$} [N^w]$ instead of $r \leftarrow_{\$} [2^{2\lambda}]$.

Claim. $\mathcal{H}_1 \approx_c \mathcal{H}_0$ assuming SEI.

Proof. Suppose there is an adversary $\mathcal{A}$ that distinguishes between $\mathcal{H}_2$ and $\mathcal{H}_1$, then we construct another adversary $\mathcal{A}'$ that distinguishes between the distribution $D_0 = \{(g, g^r) : r \leftarrow [2^{2\lambda}]\}$ and the full exponent distribution $D_1 = \{(g, g^r) : r \leftarrow [N^w]\} \approx_s (g, g') : g' \leftarrow \mathbb{G}$, which is statistically close to a random element of $\mathbb{G}$, i.e., $2N^w$ -th residue. The idea is that once $\mathcal{A}'$ receives a challenge $(g, h) \leftarrow_{\$} D_b$, it samples a secret key $s \leftarrow_{\$} S_{\text{sk}}$ and simulates a ciphertext as $\text{ct} = (h, (N+1)^{xh^{-s}})$. Denote $\mathcal{D}_{\text{ct}, b'}$ as the ciphertext distribution when $b = b'$. Note $\mathcal{D}_{\text{ct}, 0} \approx_p \{(g^r, (N+1)^{x f^r}) : r \leftarrow_{\$} [2^{2\lambda}]\}$, i.e., when $b = 0$, the distribution is identical to the actual ciphertext distribution when the random exponent r is short. Moreover, $\mathcal{D}_{\text{ct}, 1} \approx_s \{(g^r, (N+1)^{x f^r}) : r \leftarrow_{\$} [N^w]\}$, i.e., when $b = 1$, the distribution is statistically close to the actual ciphertext distribution when r is sampled from $[N^w]$. Therefore, the advantage of $\mathcal{A}'$ in SEI will then be identical to the advantage of $\mathcal{A}$ in distinguishing between $\mathcal{H}_2$ and $\mathcal{H}_1$. $\square$

- *Hybrid $\mathcal{H}_2$.* In this hybrid, we change how the public key f is sampled in `DJEG.KeyGen` by sampling $f \leftarrow_{\$} \mathbb{G}$ as a uniformly random $2N^w$ -th residue. That is, $f := (g')^{2N^w} \in \mathbb{Z}_{N^{w+1}}^*$, where $g' \leftarrow_{\$} \mathbb{Z}_{N^{w+1}}^*$.

Claim. $\mathcal{H}_2 \approx_c \mathcal{H}_1$ by Equation (1).

Proof. By the definition of $g = (g_0)^{2N^w}$, it is a generator for the subgroup of the $2N^w$ -th residues with overwhelming probability (again, using the fact that N is a composite of safe primes). Therefore, Equation (1) allows us to replace g^s with a random element $g' \leftarrow_{\$} \mathbb{G}$. Here, $(g, f) = (g, g^{-s}) \approx_c (g, (g')^{-1}) \approx_p (g, g'')$ where g'' is another uniformly random $2N^w$ -th residue. $\square$

- *Hybrid $\mathcal{H}_3$.* In this hybrid, we change how the public key f is sampled in `DJEG.KeyGen` by sampling f as a uniformly random square from $\mathbb{Z}_{N^{w+1}}^*$.

Claim. $\mathcal{H}_3 \approx_c \mathcal{H}_2$ assuming DCR.

Proof. The claim follows from a direct reduction to the DCR assumption. For simplicity, we focus on the $w = 1$ case which generalizes to $w > 1$ via induction [DJ01, Theorem 1]. Notice that f is sampled as a $2N$ -th residue in $\mathcal{H}_2$ and a random square of $\mathbb{Z}_{N^2}^*$ in $\mathcal{H}_3$. The reduction thus has at most a factor of two loss in advantage in the DCR game. $\square$

Remark 6. We note that, thanks to CRT decomposition, $\mathbb{Z}_{N^w \cdot \phi(N)/4}$ is isomorphic to $\mathbb{Z}_{N^w} \times \mathbb{Z}_{\phi(N)/4}$ because N is coprime to $\phi(N)$. Using this, any square c in $\mathbb{Z}_{N^{w+1}}^*$ can be written as $c = (1 + N)^a g^b \bmod N^{w+1}$, for some $(a, b) \in \mathbb{Z}_{N^w} \times \mathbb{Z}_{\phi(N)/4}$, since all elements in $\mathbb{Z}_{N^w \cdot \phi(N)}^*$ can be decomposed into this form. Moreover, for a random c , with overwhelming probability $1 - \frac{p+q-1}{N}$, we have that $a \neq 0$ and coprime to N .

- *Hybrid $\mathcal{H}_4$.* In this hybrid, the ciphertext elements c_0 and c_1 are sampled as uniformly random elements of $\mathbb{Z}_{N^{w+1}}^*$.

Claim. $\mathcal{H}_4 \approx_s \mathcal{H}_3$.

Proof. We claim that, in hybrid $\mathcal{H}_3$, (c_0, c_1) is already statistically close to the uniform distribution over $\mathbb{Z}_{N^{w+1}}^* \times \mathbb{Z}_{N^{w+1}}^*$. To see this, we first note that g generates a subgroup of order $\phi(N)/4$ and, therefore, the element c_0 *statistically* reveals only the value $r_0 = r \bmod \phi(N)/4$. Moreover, using Remark 6, c_1 can be rewritten as follows:

$$c_1 = (1 + N)^x \cdot f^r = (1 + N)^{ar_1 + x \bmod N^w} \cdot g^{b \cdot r_0 \bmod \phi(N)/4} \bmod N^{w+1},$$

where $r_0 = r \bmod \phi(N)/4$ and $r_1 = r \bmod N^w$. Then, conditioned on r_0 , r_1 is statistically close to a uniformly random element by the fact that N and $\phi(N)$ are coprime. By the above, we have that $ar_1 + x \bmod N^w$ is statistically close to a uniformly random element of $\mathbb{Z}_{N^w}$ given r_0 (recall that a is coprime to N , with overwhelming probability). Combined, we have that (c_0, c_1) are statistically close to a uniformly random tuple of elements sampled from $\mathbb{Z}_{N^{w+1}}^*$. $\square$

Since all ciphertexts are indistinguishable from random tuples, they are indistinguishable from each other, so this concludes the proof of semantic security for the DJEG scheme when the ciphertext is generated using `DJEG.Encrypt`. We note that a very similar hybrid argument applies to proving that ciphertexts output by the “flipped” encryption `DJEG.FlipEncrypt` are computationally indistinguishable from uniform under the DCR assumption. A little more formally, starting with $\mathcal{H}_3$, the element f is distributed identically to g (both are random squares in $\mathbb{Z}_{N^2}^*$) which enables interchanging them in c_0 and c_1 . This concludes the proof. $\blacksquare$

4.3 Distributed Discrete Log

The distributed discrete log procedure [OSY21,RS21] $\text{DDLog}_{N,w}(\cdot)$ converts a *divisive* share of $(1+N)^x \in \mathbb{Z}_{N^{w+1}}^*$ (i.e., one party holds a group element $g' \in \mathbb{Z}_{N^{w+1}}^*$ and the other holds $g'(1+N)^x$) to a subtractive share of x over the additive group $\mathbb{Z}_{N^w}$.

Below, we provide an algorithm using the subroutine $L_w(\cdot)$, which efficiently computes the discrete log with base $1+N$. $(\cdot)^{-1}$ denotes the multiplicative inverse over $\mathbb{Z}_{N^{w+1}}^*$.

$$\text{DDLog}_{N,w}(h \in \mathbb{Z}_{N^{w+1}}^*) := L_w(h(h \bmod N)^{-1} \bmod N^{w+1}). \quad (2)$$

To prove correctness, we first show a simple lemma,

Lemma 7. *For all $f \in \mathbb{Z}_{N^{w+1}}^*$, $f \equiv_N 1$ if and only if there exists some $z \in \mathbb{Z}_{N^w}$ such that $f \equiv_{N^{w+1}} (1+N)^z$, i.e., $L_w(f)$ is defined.*

Proof. The leftward implication is trivial as $(1+N)^z \equiv_N 1$. To show the other direction, we use a counting argument: there are only N^w elements $f \in \mathbb{Z}_{N^{w+1}}^*$ such that $f \equiv_N 1$, and also exactly N^w distinct values of $(1+N)^z \bmod N^{w+1}$, i.e., one for each distinct value of $z \in \mathbb{Z}_{N^w}$. Therefore, there cannot exist some $f \equiv_N 1$ that is not a power of $1+N$, since that combined with the trivial leftward implication means that there are at least $N^w + 1$ values of $f \equiv_N 1$. ■

Theorem 8. *The algorithm described in Equation (2) is a correct distributed discrete log procedure. Specifically, for any $g' \in \mathbb{Z}_{N^{w+1}}^*$ and $x \in [N^w]$, we have:*

$$\text{DDLog}_{N,w}(g'(1+N)^x) - \text{DDLog}_{N,w}(g') \equiv x \pmod{N^w}.$$

Proof. Observe that $g'(g' \bmod N)^{-1} \equiv_N (g' \bmod N)(g' \bmod N)^{-1} \equiv_N 1$, where the first equivalence follows because the multiplicative inverse over $\mathbb{Z}_{N^w}^*$ is also an inverse over $\mathbb{Z}_N^*$ as N divides N^w . This, along with Lemma 7, shows that for all $g' \in \mathbb{Z}_{N^{w+1}}^*$, $g'(g' \bmod N)^{-1} = (1+N)^z$ for some z . Then, we can verify correctness:

$$\begin{aligned} & \text{DDLog}_{N,w}(g'(1+N)^x) - \text{DDLog}_{N,w}(g') \\ & \equiv L_w(g'(1+N)^x(g' \bmod N)^{-1}) - L_w(g'(g' \bmod N)^{-1}) \\ & \equiv L_w((1+N)^{x+z}) - L_w((1+N)^z) \\ & \equiv x + z - z \\ & \equiv x \pmod{N^w}. \end{aligned}$$

■

In our MKHSS construction, we commonly require that subtractive shares be over the *integers*, rather than over N^w , so two parties often apply a common random shift $r \in [N^w]$ to the output of DDLog to obtain a subtractive share over $\mathbb{Z}$ with high probability over r : $\langle z \rangle_\sigma \leftarrow \text{DDLog}_{N,w}(g^{(0)\sigma}(1+N)^{\langle z \rangle_\sigma}) + r \bmod N^w$. To see correctness, first note that we can define $g' = g^{(0)B}(1+N)^{\langle z \rangle_B}$ and $x = z$ and invoke Theorem 8. Then, over the random shift r , $\langle z \rangle_\sigma$ is a valid subtractive share over $\mathbb{Z}$ with probability at least $1 - 2^{-\tau}$ assuming that the absolute value of z is bounded: $|z| \leq N^w 2^{-\tau}$.

4.4 Non-Interactive Multiplication

Non-interactive multiplication (NIM) [BDSS25] allows Alice and Bob, holding t_A and t_B , to exchange a simultaneous message and each obtain a subtractive share of the product $\langle t_A t_B \rangle_\sigma$. The communication pattern is non-interactive and similar to ANIKE (Figure 1). We give a formal definition below, adapted from [BDSS25] with the notable difference being that Decode takes in an additional *common* random coin between two parties:

Definition 9 (Non-Interactive Multiplication; Adapted from [BDSS25]). *Let λ be a security parameter. A non-interactive multiplication (NIM) scheme consists of three algorithms $\text{NIM} = (\text{Setup}, \text{Encode}, \text{Decode})$ along with a random coin distribution $\mathcal{R}$ and an integer bound B with the following syntax:*

- $\text{Setup}(1^\lambda) \rightarrow \text{crs}$. The randomized setup algorithm takes as input the security parameter and outputs a common reference string crs .
- $\text{Encode}(\text{crs}, \sigma, t_\sigma) \rightarrow (\text{pe}_\sigma, \text{st}_\sigma)$. The randomized encoding algorithm takes as input the CRS crs , a party identifier σ , and a B -bounded integer $t_\sigma \in [-B, B]$. It outputs a public encoding pe_σ and secret state st_σ .
- $\text{Decode}(\text{crs}, \text{pe}_{1-\sigma}, \text{st}_\sigma; r_{\text{nim}}) \rightarrow \langle z \rangle_\sigma$. The deterministic decoding algorithm takes as input the CRS crs , another party's public encoding $\text{pe}_{1-\sigma}$, secret state st_σ , and some common randomness $r_{\text{nim}} \in \mathcal{R}$ that must be the same between Alice and Bob. It outputs a subtractive secret share of $z = t_A t_B$ with high probability over r_{nim} .

The above functionality must satisfy correctness and security, which are defined as follows:

Correctness. For all security parameters $\lambda \in \mathbb{N}$, statistical security parameters $\tau \in \mathbb{N}$, and every pair of elements $t_A, t_B \in \mathbb{Z}$ such that $t_A t_B \in [-B, B]$, a NIM scheme is said to be correct if there exists a negligible function $\text{negl}(\cdot)$ such that:

$$\Pr \left[\begin{array}{l} \langle z \rangle_A - \langle z \rangle_B = t_A t_B \quad : \quad \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda, 1^\tau) \\ r_{\text{nim}} \leftarrow \mathcal{R} \\ (\text{pe}_A, \text{st}_A) \leftarrow \text{Encode}(\text{crs}, A, t_A) \\ (\text{pe}_B, \text{st}_B) \leftarrow \text{Encode}(\text{crs}, B, t_B) \\ \langle z \rangle_A := \text{Decode}(\text{crs}, \text{pe}_B, \text{st}_A; r_{\text{nim}}) \\ \langle z \rangle_B := \text{Decode}(\text{crs}, \text{pe}_A, \text{st}_B; r_{\text{nim}}) \end{array} \end{array} \right] \geq 1 - \text{negl}(\tau).$$

Security. A NIM scheme is said to be secure if for all efficient adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$, and all $\sigma \in \{A, B\}$, we have that

$$\Pr \left[\begin{array}{l} b' = b \quad : \quad \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda) \\ (t_0, t_1, \text{state}) \leftarrow \mathcal{A}(\text{crs}) \\ b \leftarrow \mathbb{Z} \{0, 1\} \\ (\text{pe}_\sigma, \text{st}_\sigma) \leftarrow \text{Encode}(\text{crs}, \sigma, t_b) \\ b' \leftarrow \mathcal{A}(\text{pe}_\sigma, \text{state}) \end{array} \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda),$$

where $t_0, t_1 \in [-B, B]$.

Constructing NIM from DCR. We give a simple construction in Construction 4 with the random coin distribution $\mathcal{R}$ being uniformly random over $[N^w]$ and bound $B = N^w 2^{-\tau}$. This construction is an adaptation of that one in Couteau et al. [CDH⁺25] to short-exponents for concrete performance benefits.

Theorem 10. Assuming DCR and $\mathbb{G}$ -SEI, where $\mathbb{G}$ denote the subgroup of $2N^w$ -th residues in $\mathbb{Z}_{N^{w+1}}^*$, Construction 4 is a non-interactive multiplication scheme.

Proof. We prove correctness and security in turn.

Correctness. Since $Z_B = c_0^{r_A} c_1^{t_A} = g^{r_A t_B} (N+1)^{t_A t_B} h^{t_A r_B} = Z_A \cdot (N+1)^{t_A t_B}$, correctness holds with probability at least $1 - 2^{-\tau}$ over r_{nim} as long as $|t_A t_B| \leq N^w 2^{-\tau} = B$.

Security ($\sigma = A$). Alice's public encoding is similar to a Pedersen commitment. We observe that $\text{Encode}(\text{crs}, A, t)$ outputs the following distribution $\mathcal{D}_t = \{g^r h^t : r \leftarrow [2^{2\lambda}]\}$. Security is equivalent to showing that $\mathcal{D}_{t_0} \approx_c \mathcal{D}_{t_1}$ for all $t_0, t_1 \in [-B, B]$, which can be shown with a sequence of hybrids:

$$\begin{aligned} \mathcal{D}_{t_0} &\approx_p \{g^r h^{t_0} : r \leftarrow [2^{2\lambda}]\} \\ &\approx_c \{g' h^{t_0} : g' \leftarrow \mathbb{G}\} \quad (\text{By SEI}) \\ &\approx_p \{g' : g' \leftarrow \mathbb{G}\} \quad (h \in \mathbb{G}) \\ &\approx_p \{g' h^{t_1} : g' \leftarrow \mathbb{G}\} \\ &\approx_c \{g^r h^{t_1} : r \leftarrow [2^{2\lambda}]\} \quad (\text{By SEI}) \\ &\approx_p \mathcal{D}_{t_1}. \end{aligned}$$

Construction 4: Non-Interactive Multiplication from DCR

Parameters. Security parameter λ , a statistical security parameter τ , and a modulus N .

NIM.Setup($1^\lambda, 1^\tau$): 1 : $g_0, h_0 \leftarrow \mathbb{Z}_{N^{w+1}}^*$ 2 : $g := (g_0)^{2^{N^w}} \bmod N^{w+1}$ 3 : $h := (h_0)^{2^{N^w}} \bmod N^{w+1}$ 4 : $\text{crs}^{\text{nim}} := (g, h)$ 5 : return crs^{nim}	NIM.Encode($\text{crs}^{\text{nim}}, \sigma, t_\sigma$): 1 : $r_\sigma \leftarrow \mathbb{Z}_{2^{2\lambda}}$ 2 : if $\sigma = A$, 3 : $\text{pe}_A^{\text{nim}} := g^{r_A} h^{t_A}$ 4 : $\text{st}_A^{\text{nim}} := (t_A, r_A)$ 5 : else , $\parallel \sigma = B$ 6 : $\text{pe}_B^{\text{nim}} := (g^{r_B}, (N+1)^{t_B} h^{r_B})$ 7 : $\text{st}_B^{\text{nim}} := r_B$ 8 : return $(\text{pe}_\sigma^{\text{nim}}, \text{st}_\sigma^{\text{nim}})$
--	--

NIM.Decode($\text{crs}, \sigma, \text{st}_{1-\sigma}^{\text{nim}}, \text{pe}_{1-\sigma}^{\text{nim}}; r_{\text{nim}}$):

- 1 : **parse** $\text{st}^{\text{nim}} = (t_\sigma, r_\sigma)$
- 2 : **if** $\sigma = A$, **parse** $\text{pe}_B^{\text{nim}} = (c_0, c_1)$
- 3 : $Z_\sigma := c_0^{r_A} c_1^{t_A}$ **if** $\sigma = A$ **else** $(\text{pe}_A^{\text{nim}})^{r_B}$
- 4 : **return** $\text{DDLog}_{N,w}(Z_\sigma) + r_{\text{nim}} \bmod N^w$

Security ($\sigma = B$). We can view Bob's public encoding as a DJEG encryption with public key h . Since g, h are generated independently at random from the same group, $(g, h) \approx_s \{(g, g^s) : s \in [N]\}$ so h is indistinguishable from a legitimate DJEG public key. Any adversary that distinguishes Bob's encoding can be immediately applied to break the semantic security of DJEG. Therefore, security of Bob's encoding follows from DCR. ■

5 Definitions

5.1 Multi-Key HSS

We define multi-key HSS in Definition 11. $\llbracket x \rrbracket^\sigma = (\llbracket x \rrbracket_A^\sigma, \llbracket x \rrbracket_B^\sigma)$ denotes an input share of a message x generated under the public key of party σ , and denote by $\{\{x\}\}$ a *synchronized* share of x which will be the same between two parties. We use $1 - \sigma$ to denote the "other party".

Definition 11 (Multi-Key Homomorphic Secret Sharing). For a positive integer B , a B -bounded multi-key homomorphic secret sharing (MKHSS) scheme is a tuple of four polynomial time algorithms $\text{MKHSS} = (\text{Setup}, \text{KeyGen}, \text{Share}, \text{Eval})$ with the following syntax:

- **Setup**($1^\lambda, 1^\tau$) $\rightarrow$ crs . On input a security parameter λ , a statistical security parameter τ , generates a reusable common reference string crs . When the statistical security parameter is not provided, we assume it is equal to the security parameter: $\text{Setup}(1^\lambda) := \text{Setup}(1^\lambda, 1^\lambda)$.
- **KeyGen**(crs) $\rightarrow$ (pk, sk) . On input the CRS crs , each party independently runs this to generate its own pair of public and secret keys.
- **Share**($\text{crs}, \sigma, \text{pk}_\sigma, x$) $\rightarrow$ $(\llbracket x \rrbracket_A^\sigma, \llbracket x \rrbracket_B^\sigma)$. On input the CRS crs , party identifier $\sigma \in \{A, B\}$, the public key pk_σ of the party, and a message $x \in [-B+1, B-1]$, output input shares $\llbracket x \rrbracket_A^\sigma$ and $\llbracket x \rrbracket_B^\sigma$. Abusing notation, we permit **Share** to be called on vectors $\mathbf{x}$.
- **Eval**($\text{crs}, \sigma, \text{sk}_\sigma, \text{pk}_{1-\sigma}, \llbracket \mathbf{x}_A \rrbracket_\sigma^A, \llbracket \mathbf{x}_B \rrbracket_\sigma^B, P$) $\rightarrow$ $\langle z \rangle_\sigma$. On input the CRS crs , a party identifier $\sigma \in \{A, B\}$, the party's secret key sk_σ , and the other party's public key $\text{pk}_{1-\sigma}$, a list of input shares from Alice $\llbracket \mathbf{x}_A \rrbracket_\sigma^A$, and respectively from Bob $\llbracket \mathbf{x}_B \rrbracket_\sigma^B$, and an RMS program description P , output a subtractive share of the output $z = P(\mathbf{x}, \mathbf{y})$.

In addition, MKHSS satisfies the following correctness and security properties:

Correctness. A B -bounded MKHSS scheme is correct if there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}, \tau \in \mathbb{N}$, all $(m_A + m_B)$ -input B -bounded RMS programs P whose memory values are in the range $[-B + 1, B - 1]$, and all $\mathbf{x}_A \in [0, B - 1]^{m_A}$ and $\mathbf{x}_B \in [0, B - 1]^{m_B}$:

$$\Pr \left[\begin{array}{l} \langle z \rangle_A - \langle z \rangle_B \\ \neq \\ P(\mathbf{x}_A, \mathbf{x}_B) \end{array} : \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda) \\ (\text{pk}_\sigma, \text{sk}_\sigma) \leftarrow \text{KeyGen}(\text{crs}), \forall \sigma \in \{A, B\} \\ (\llbracket \mathbf{x}_\sigma \rrbracket_A^\sigma, \llbracket \mathbf{x}_\sigma \rrbracket_B^\sigma) \leftarrow \text{Share}(\text{crs}, \sigma, \text{pk}_\sigma, \mathbf{x}_\sigma), \forall \sigma \in \{A, B\} \\ \langle z \rangle_\sigma \leftarrow \text{Eval}(\text{crs}, \sigma, \text{sk}_\sigma, \text{pk}_{1-\sigma}, \llbracket \mathbf{x}_A \rrbracket_\sigma^A, \llbracket \mathbf{x}_B \rrbracket_\sigma^B, P), \forall \sigma \in \{A, B\} \end{array} \right] \leq \varepsilon + \text{negl}(\lambda),$$

where $\mathbf{x}_\sigma = (x_\sigma^{(1)}, \dots, x_\sigma^{(m)})$.

Security. An MKHSS scheme is said to be secure if for all efficient adversaries $\mathcal{A}$, there exists a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$, and all $\sigma \in \{A, B\}$, we have that

$$\Pr \left[\begin{array}{l} b' = b : \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda) \\ (\text{pk}_\sigma, \text{sk}_\sigma) \leftarrow \text{KeyGen}(\text{crs}) \\ (x_0, x_1, \text{state}) \leftarrow \mathcal{A}(\text{crs}, \text{pk}_\sigma) \\ b \leftarrow \{0, 1\} \\ (\llbracket x_b \rrbracket_A^\sigma, \llbracket x_b \rrbracket_B^\sigma) \leftarrow \text{Share}(\text{crs}, \sigma, \text{pk}_\sigma, x_b) \\ b' \leftarrow \mathcal{A}(\llbracket x_b \rrbracket_{1-\sigma}^\sigma, \text{state}) \end{array} \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

Remark 12 (Extension of security to multiple shares). It is straightforward to show that the security of MKHSS implies security under a modified game where x_0 and x_1 are replaced with two vectors $\mathbf{x}_0$ and $\mathbf{x}_1$ via a hybrid argument.

5.2 Attribute-Based Non-Interactive Key Exchange

We define attribute-based non-interactive key exchange in Definition 13. The definition is almost the same as in Couteau et al. [CDH⁺25], except we allow asymmetric policies and fix the policy P as a parameter.

Definition 13 (Attribute-Based Non-Interactive Key Exchange). An attribute-based non-interactive key exchange (ANIKE) protocol with attribute spaces $\mathcal{X}_A, \mathcal{X}_B$ and a policy $P : \mathcal{X}_A \times \mathcal{X}_B \rightarrow \{0, 1\}$ consists of three efficient algorithms (Setup, AttrKeyGen, AttrKeyDer) with the following syntax:

- $\text{Setup}(1^\lambda) \rightarrow \text{crs}$. On input a security parameter, generates a common reference string (CRS) crs .
- $\text{AttrKeyGen}(\text{crs}, \sigma, x) \rightarrow (\text{pe}_\sigma, \text{st}_\sigma)$. On input the CRS crs , party identifier $\sigma \in \{A, B\}$, and attribute $x \in \mathcal{X}_\sigma$, outputs the public encoding pe_σ and private state st_σ .
- $\text{AttrKeyDer}(\text{crs}, \sigma, \text{st}_\sigma, \text{pe}_{1-\sigma}) \rightarrow k_\sigma$. On input the CRS crs , the party identifier $\sigma \in \{A, B\}$, the party's own private state st_σ , the other party's public encoding $\text{pe}_{1-\sigma}$, outputs a key $k_\sigma \in \{0, 1\}^\lambda$.

In addition, ANIKE satisfies the following correctness and semi-honest security properties.

Correctness. An ANIKE scheme is correct if there is a negligible function $\text{negl}(\cdot)$ such that for all $\lambda \in \mathbb{N}$, $x_A \in \mathcal{X}_A$, and $x_B \in \mathcal{X}_B$ such that $P(x_A, x_B) = 0$, i.e., the policy is satisfied, we have

$$\Pr \left[\begin{array}{l} k_A = k_B : \begin{array}{l} \text{crs} \leftarrow \text{Setup}(1^\lambda) \\ (\text{pe}_A, \text{st}_A) \leftarrow \text{AttrKeyGen}(\text{crs}, A, x_A) \\ (\text{pe}_B, \text{st}_B) \leftarrow \text{AttrKeyGen}(\text{crs}, B, x_B) \\ k_A \leftarrow \text{AttrKeyDer}(\text{crs}, A, \text{st}_A, \text{pe}_B) \\ k_B \leftarrow \text{AttrKeyDer}(\text{crs}, B, \text{st}_B, \text{pe}_A) \end{array} \end{array} \right] \leq \frac{1}{2} + \text{negl}(\lambda).$$

Semi-honest security. An ANIKE scheme is secure against semi-honest adversaries if it satisfies the three properties—key pseudorandomness, key independence, and attribute hiding—defined through security experiments E^{KP} , E^{KI} , and E^{AH} , respectively—in Figure 2.

Parameters. $\bar{\sigma}$ denotes the opposite party of σ . Each security property P is satisfied if for all $\sigma \in \{A, B\}$ and all efficient adversaries $\mathcal{A}$, $\Pr[\mathbf{E}_{\mathcal{A}}^P(\lambda, \sigma) = 1] \leq \text{negl}(\lambda)$.

$\mathbf{E}_{\mathcal{A}}^{\text{KP}}(\lambda)$

// Key pseudorandomness experiment

$\text{crs} \leftarrow \text{Setup}(1^\lambda)$

$b \leftarrow_{\$} \{0, 1\}$

$(x_A, x_B, \text{state}) \leftarrow \mathcal{A}(\text{crs})$

if $P(x_A, x_B) \neq 0$, **return** 0

$(\text{pe}_A, \text{st}_A) \leftarrow \text{AttrKeyGen}(\text{crs}, A, x_A)$

$(\text{pe}_B, \text{st}_B) \leftarrow \text{AttrKeyGen}(\text{crs}, B, x_B)$

$k_0 \leftarrow \text{AttrKeyDer}(\text{crs}, B, \text{st}_B, \text{pe}_A)$

$k_1 \leftarrow_{\$} \{0, 1\}^\lambda$

return $\mathbb{1}(\mathcal{A}(k_b, \text{pe}_A, \text{pe}_B, \text{state}) = b)$

$\mathbf{E}_{\mathcal{A}}^{\text{KI}}(\lambda, \sigma \in \{A, B\})$

// Key independence experiment

$\text{crs} \leftarrow \text{Setup}(1^\lambda)$

$b \leftarrow_{\$} \{0, 1\}$

$(x_A, x_B, \text{state}) \leftarrow \mathcal{A}(\text{crs}, \sigma)$

if $P(x_A, x_B) \neq 1$, **return** 0

$r_\sigma \leftarrow_{\$}$ Random coins for AttrKeyGen

$(\text{pe}_\sigma, -) := \text{AttrKeyGen}(\text{crs}, \sigma, x_\sigma; r_\sigma)$

$(\text{pe}_{\bar{\sigma}}, \text{st}_{\bar{\sigma}}) \leftarrow \text{AttrKeyGen}(\text{crs}, \bar{\sigma}, x_{\bar{\sigma}})$

$k_0 \leftarrow \text{AttrKeyDer}(\text{crs}, \bar{\sigma}, \text{st}_{\bar{\sigma}}, \text{pe}_\sigma)$

$k_1 \leftarrow_{\$} \{0, 1\}^\lambda$

return $\mathbb{1}(\mathcal{A}(k_b, \text{pe}_{\bar{\sigma}}, r_\sigma, \text{state}) = b)$

$\mathbf{E}_{\mathcal{A}}^{\text{AH}}(\lambda, \sigma \in \{A, B\})$

// Attribute hiding experiment

$\text{crs} \leftarrow \text{Setup}(1^\lambda)$

$b \leftarrow_{\$} \{0, 1\}$

$(x_0, x_1, \text{state}) \leftarrow \mathcal{A}(\text{crs}, \sigma)$

$(\text{pe}_\sigma, -) := \text{AttrKeyGen}(\text{crs}, \sigma, x_b)$

return $\mathbb{1}(\mathcal{A}(\text{pe}_\sigma, \text{state}) = b)$

Fig. 2: Security experiments for ANIKE.

6 Full Multi-Key HSS Construction

Construction 5: Our MKHSS scheme

Parameters. Security parameter λ , statistical security parameter τ , bit length of the modulus l_N , RMS program bound B , and DJEG parameter w . $\text{GenPQ}(1^\lambda)$ generates two $\lceil l_N/2 \rceil$ -bit safe primes. $\text{DDLog}_{N,w}$ is defined in Section 4.3. $\text{NIM} = (\text{NIM.Setup}, \text{NIM.Encode}, \text{NIM.Decode})$ is defined in Section 4.4. H is a random oracle. Let $G_m : \{0, 1\}^\lambda \rightarrow \mathbb{Z}_m$ be a pseudorandom generator, and $F_m : \{0, 1\}^\lambda \times \mathbb{Z} \rightarrow \mathbb{Z}_m$ be a pseudorandom function with output space $\mathbb{Z}_m$. The subroutines InitRMS and DEval are defined in Figure 3.

MKHSS.Setup($1^\lambda, 1^\tau$):

```

1 :  $(p, q) \leftarrow \text{GenPQ}(1^\lambda)$ 
2 :  $N := pq$ 
3 :  $g_0 \leftarrow \mathbb{Z}_{N^{w+1}}^*$ 
4 :  $g := (g_0)^{2N^w} \bmod N^{w+1}$ 
5 :  $\text{crs}^{\text{nim}} := \text{NIM.Setup}(1^\lambda)$ 
6 :  $\text{crs} := (N, g, \text{crs}^{\text{nim}})$ 
7 : return  $\text{crs}$ 
```

MKHSS.KeyGen(crs):

```

1 : parse  $\text{crs} = (N, g, \text{crs}^{\text{nim}})$ 
2 :  $t \leftarrow \mathbb{Z}_{2^{2\lambda}}$ 
3 :  $s := t \cdot (B \cdot 2^\tau) + 1$ 
4 :  $f := g^{-s} \bmod N^{w+1}$ 
5 :  $(\text{pe}_A^{\text{nim}}, \text{st}_A^{\text{nim}}) \leftarrow \text{NIM.Encode}(\text{crs}^{\text{nim}}, A, t)$ 
6 :  $(\text{pe}_B^{\text{nim}}, \text{st}_B^{\text{nim}}) \leftarrow \text{NIM.Encode}(\text{crs}^{\text{nim}}, B, t)$ 
7 :  $\text{pk} := (f, \text{pe}_A^{\text{nim}}, \text{pe}_B^{\text{nim}})$ 
8 :  $\text{sk} := (s, \text{st}_A^{\text{nim}}, \text{st}_B^{\text{nim}})$ 
9 : return  $(\text{pk}, \text{sk})$ 
```

MKHSS.Share($\text{crs}, \sigma, \text{pk}, x$):

```

1 : parse  $\text{crs} = (N, g, -)$ 
2 : parse  $\text{pk} = (f, -, -)$ 
3 :  $r \leftarrow \mathbb{Z}_{2^{2\lambda}}$ 
4 :  $\text{ct}_0 := (N + 1)^x g^r \bmod N^{w+1}$ 
5 :  $\text{ct}_1 := f^r \bmod N^{w+1}$ 
6 :  $\llbracket x \rrbracket_\sigma^\sigma := (r, \text{ct}_0)$ 
7 :  $\llbracket x \rrbracket_{1-\sigma}^\sigma := (\text{ct}_0, \text{ct}_1)$ 
8 : return  $(\llbracket x \rrbracket_A^\sigma, \llbracket x \rrbracket_B^\sigma)$ 
```

MKHSS.Eval($\text{crs}, \sigma, \text{sk}_\sigma, \text{pk}_{1-\sigma}, \llbracket \mathbf{x}_A \rrbracket_\sigma^A, \llbracket \mathbf{x}_B \rrbracket_\sigma^B, P$):

```

1 : parse  $\text{crs} = (N, g, -)$ 
2 : parse  $\llbracket \mathbf{x}_A \rrbracket_\sigma^A = \llbracket x_A^{(1)} \rrbracket_\sigma^A, \dots, \llbracket x_A^{(m_A)} \rrbracket_\sigma^A$ 
3 : parse  $\llbracket \mathbf{x}_B \rrbracket_\sigma^B = \llbracket x_B^{(1)} \rrbracket_\sigma^B, \dots, \llbracket x_B^{(m_B)} \rrbracket_\sigma^B$ 
4 :  $\text{ek}_\sigma := \text{InitRMS}(\text{crs}, \sigma, \text{sk}_\sigma, \text{pk}_{1-\sigma}, \llbracket \mathbf{x}_A \rrbracket_\sigma^A, \llbracket \mathbf{x}_B \rrbracket_\sigma^B)$ 
5 : for  $i \in [m_\sigma]$ , // Sync one's own shares
6 :  $\{\{x_\sigma^{(i)}\}\} := \text{ExpLinEncS}(\text{sk}_\sigma, \text{ek}_\sigma, \llbracket x_\sigma^{(i)} \rrbracket_\sigma^\sigma)$ 
7 : for  $i \in [m_{1-\sigma}]$ , // Sync received shares
8 :  $\{\{x_{1-\sigma}^{(i)}\}\} := \text{ExpLinEncR}(\text{sk}_\sigma, \text{pk}_{1-\sigma}, \llbracket x_{1-\sigma}^{(i)} \rrbracket_\sigma^{1-\sigma})$ 
9 :  $\{\{\mathbf{x}\}\} := \{\{x_A^{(1)}\}\}, \dots, \{\{x_A^{(m_A)}\}\}, \{\{x_B^{(1)}\}\}, \dots, \{\{x_B^{(m_B)}\}\}$ 
10 : return  $\text{DEval}(\sigma, \text{ek}_\sigma, \{\{\mathbf{x}\}\}, P)$ 
```

ExpLinEncS($\text{sk}_\sigma, \text{ek}_\sigma, \llbracket x_\sigma^{(i)} \rrbracket_\sigma^\sigma$):

```

1 : parse  $\text{ek}_\sigma = (-, -, f)$ 
2 : parse  $\text{sk}_\sigma = (s_\sigma, -)$ 
3 : parse  $\llbracket x_\sigma^{(i)} \rrbracket_\sigma^\sigma = (r, \text{ct}_0)$ 
4 :  $\{\{x_\sigma^{(i)}\}\} := (\text{ct}_0, f^r)$ 
5 : return  $\{\{x_\sigma^{(i)}\}\}$ 
```

ExpLinEncR($\text{sk}_\sigma, \text{pk}_{1-\sigma}, \llbracket x_{1-\sigma}^{(i)} \rrbracket_\sigma^{1-\sigma}$):

```

1 : parse  $\text{sk}_\sigma = (s_\sigma, -)$ 
2 : parse  $\text{pk}_{1-\sigma} = (f_{1-\sigma}, -, -)$ 
3 : parse  $\llbracket x_{1-\sigma}^{(i)} \rrbracket_\sigma^{1-\sigma} = (\text{ct}_0, \text{ct}_1)$ 
4 :  $\{\{x_{1-\sigma}^{(i)}\}\} := (\text{ct}_0, \text{ct}_1^{s_\sigma})$ 
5 : return  $\{\{x_{1-\sigma}^{(i)}\}\}$ 
```

We describe our MKHSS construction in Construction 5.

Theorem 14. Assuming DCR and the hardness of short exponent discrete log (SEDL) over the group of $2N^w$ -th residues over $\mathbb{Z}_{N^{w+1}}^*$, our the construction in Construction 5 is a correct and secure MKHSS scheme.

Proof. We prove correctness and security in turn.

Fig. 3: Additional subroutines used by our MKHSS construction.

Parameters. See Construction 5.

```

InitRMS(crs,  $\sigma$ ,  $\text{sk}_\sigma$ ,  $\text{pk}_{1-\sigma}$ ,  $\llbracket \mathbf{x}_A \rrbracket_\sigma^A$ ,  $\llbracket \mathbf{x}_B \rrbracket_\sigma^B$ ):
  1 : parse  $\text{sk}_\sigma = (s_\sigma, \text{st}^{\text{nim}})$ 
  2 : parse  $\text{pk}_{1-\sigma} = (f_{1-\sigma}, \text{pe}_A^{\text{nim}}, \text{pe}_B^{\text{nim}})$ 
  3 :  $f := f_{1-\sigma}^{s_\sigma}$  // Precomputed  $f$  speeds up ExpLinEncS
  4 :  $k_{\text{rms}} \parallel k_{\text{nim}} := H(f)$ 
  5 :  $\langle t_A t_B \rangle'_\sigma := \text{NIM.Decode}(\text{crs}, \sigma, \text{st}^{\text{nim}}, \text{pe}_{1-\sigma}^{\text{nim}}; G_{N^w}(k_{\text{nim}}))$ 
  6 :  $\langle t_A t_B \rangle_\sigma := \langle t_A t_B \rangle'_\sigma \bmod 2^{4\lambda+\tau}$ 
  7 :  $\langle 1 \rangle_\sigma := 1$  if  $\sigma = A$  else  $0$ 
  8 :  $\langle t_A + t_B \rangle_\sigma := t_A$  if  $\sigma = A$  else  $-t_B$ 
  9 :  $\langle s \rangle_\sigma := \langle t_A t_B \rangle_\sigma (B')^2 + \langle t_A + t_B \rangle_\sigma B' + \langle 1 \rangle_\sigma$ 
  10 :  $\langle 1 \rangle_\sigma := (\langle s \rangle_\sigma, \langle 1 \rangle_\sigma)$ 
  11 :  $\text{ek}_\sigma := (k_{\text{rms}}, \langle 1 \rangle_\sigma, f)$ 
  12 : return  $\text{ek}_\sigma$ 

DEval( $\sigma, \text{ek}_\sigma, (\{x_1\}, \dots, \{x_m\}), P$ ):
  Parse  $\text{ek}_\sigma = (k_{\text{rms}}, \langle 1 \rangle_\sigma, f)$ .
  For each  $\text{id} \in [P]$ , evaluate the  $\text{id}$ -th instruction:
    • Convert:  $M_x \leftarrow I_x$ :
      1: Execute  $\langle x \rangle_\sigma \leftarrow \text{Mult}(\{x\}, \langle 1 \rangle_\sigma)$ .
    • Mult:  $M_{xy} \leftarrow I_x \cdot M_y$ :
      1: Parse  $\{x\} = (\text{ct}_0, \text{ct}_1)$ .
      2: Parse  $\langle y \rangle_\sigma = (\langle y \cdot s \rangle_\sigma, \langle y \rangle_\sigma)$ .
      3:  $\langle xys \rangle'_\sigma \leftarrow \text{DDLog}_{N,w}(\text{ct}_0^{\langle y \cdot s \rangle_\sigma} \text{ct}_1^{\langle y \rangle_\sigma}; F_{N^w}(k_{\text{rms}}, \text{id}))$ .
      4:  $\langle xys \rangle_\sigma \leftarrow \langle xys \rangle'_\sigma \bmod B^3 \cdot 2^{4\tau+3\lambda}$ .
      5:  $\langle xy \rangle_\sigma \leftarrow \langle xys \rangle_\sigma \bmod B \cdot 2^\tau$ .
      6:  $\langle xy \rangle_\sigma := (\langle xys \rangle_\sigma, \langle xy \rangle_\sigma)$ 
    • ScaleAddM:  $M_{ax+by} \leftarrow aM_x + bM_y$ :
      1:  $\langle ax + by \rangle_\sigma := a\langle x \rangle_\sigma + b\langle y \rangle_\sigma$ .
    • ScaleAddI:  $I_{ax+by} \leftarrow aI_x + bI_y$ :
      1:  $\{ax + by\} := \{x\}^a \cdot \{y\}^b$ .
    • Output:  $z \leftarrow M_z$ :
      1: Parse  $\langle z \rangle_\sigma = (\langle z \cdot s \rangle_\sigma, \langle z \rangle_\sigma)$ .
      2: Return  $\langle z \rangle_\sigma$ .

```

Correctness. Recall that the correctness property requires that parties obtain a subtractive sharing of the program output upon evaluation.

We give an outline of the correctness proof:

1. We first prove that the encoding $\{\{x\}\}$ derived by the parties in `MKHSS.Eval` is the same for both parties, specifically of the form $\{\{x\}\} = ((N+1)^x \cdot g^r, f^r)$, where $f = g^{-s_{AB}}$ is the joint public key.
2. We then show that `InitRMS` outputs a valid ek_σ , specifically showing that the second component of ek_σ is actually a valid memory share of one $\langle\langle 1 \rangle\rangle_\sigma$.
3. Finally, we show that `DEval` correctly outputs subtractive shares of the output of the RMS program.

Claim. For all integers $x \in \mathbb{Z}_N$ and all $\sigma \in \{A, B\}$, we have

$$\{\{x\}\} = \text{ExpLinEncS}(\text{sk}_\sigma, \text{ek}_\sigma, \llbracket x \rrbracket_\sigma^\sigma) = \text{ExpLinEncR}(\text{sk}_{1-\sigma}, \text{pk}_\sigma, \llbracket x \rrbracket_{1-\sigma}^\sigma),$$

where $(\llbracket x \rrbracket_A^\sigma, \llbracket x \rrbracket_B^\sigma) \leftarrow \text{MKHSS.Share}(\text{crs}, \sigma, \text{pk}_\sigma, x)$ and $\text{ek}_\sigma := (-, -, f)$. Moreover, $\{\{x\}\}$ decrypts to xs using the standard DJEG decryption procedure using the joint secret key $s = s_{AB}$.

Proof. We consider the case where $\sigma = A$; a symmetric argument follows for the case where $\sigma = B$.

By inspecting `MKHSS.Share`, we have $\llbracket x \rrbracket_A^A = (r, \text{ct}_0)$ and $\llbracket x \rrbracket_B^A = (\text{ct}_0, \text{ct}_1)$, where $\text{ct}_0 = (N+1)^x \cdot g^r$ and $\text{ct}_1 = f_A^r$.

Party-*A* computes $\{\{x\}\}$ in `ExpLinEncS` as $\{\{x\}\} = ((N+1)^x \cdot g^r, f^r)$. Now, observe that party-*B* computes $\{\{x\}\}$ in `ExpLinEncR` as:

$$\begin{aligned} \{\{x\}\} &= ((N+1)^x \cdot g^r, (f_A^r)^{s_B}) \\ &= ((N+1)^x \cdot g^r, (f^{s_B})^r) \\ &= ((N+1)^x \cdot g^r, f^r). \end{aligned}$$

Therefore, both parties obtain the same encoding $\{\{x\}\}$. □

The correctness of `InitRMS` is mostly self-evident from the description of the algorithm except line 6, which claims that local reduction modulo $2^{4\lambda+\tau}$ gives a new subtractive share of $t_A t_B$. The correctness of this line follows from the fact that `MKHSS.KeyGen` ensures that $t_\sigma < 2^{2\lambda}$ for $\sigma \in \{A, B\}$, so $t_A t_B < 2^{4\lambda}$. Since `NIM.Decode` outputs a *random* subtractive share (ensured by the PRG output $G_{N^w}(k_{\text{nim}})$), the probability that reducing mod $2^{4\lambda+\tau}$ fails to maintain subtractive shares is less than $2^{-\tau} \leq \text{negl}(\tau)$.

We finally show that `DEval` produces a valid subtractive share of the output $P(\mathbf{x})$. Let v_i denote the i -th value (either input or memory) produced by the RMS program P , where $v_0 = 1$, i.e., the initial memory share of 1. We show by induction that the following invariant is maintained: after i RMS instructions, for every $0 \leq j \leq i$:

1. If v_i is a memory value, then $\langle\langle v_j \rangle\rangle_\sigma := (\langle v_j s \rangle_\sigma, \langle v_j \rangle_\sigma)$ represent a valid memory share of v_j for all $0 \leq j \leq i$.
2. If v_i is an input value, then $\{\{v_j\}\}$ is a valid synchronized input share, specifically it is a ciphertext of the format: $((N+1)^{v_j} \cdot g^r, f^r)$.

The base case follows from the validity of ek_σ , which subsequently follows from the correctness of `InitRMS`. We then show that `DEval` handles all instructions correctly:

1. **ScaleAdd_M**: The instruction linearly maps two memory shares to a new memory share. It implements this operation by component-wise linear operations on the two subtractive shares inside a memory share. Correctness follows from the fact that subtractive shares are linear: local linear operations translates to linear operations on the shared value.
2. **ScaleAdd_I**: The instruction linearly maps two input shares to a new input share. It implements this operation by translating scalar multiplication to component-wise exponentiation and addition to component-wise multiplication. Correctness (i.e., the output decrypts to $(ax + by)s = a(xs) + b(ys)$) follows from the linearly homomorphic property of DJEG.

3. **Mult**: This RMS multiplication instruction maps an input share of x and a memory share of y into a memory share of xy .

Let $r_i = F_{N^w}(k_{\text{rms}}, i)$. We first show that $\langle xy \rangle'_\sigma$ is a valid subtractive share:

$$\begin{aligned} \langle xy \rangle'_\sigma &= \text{DDLog}_{N,w}(\text{ct}_0^{\langle y \cdot s \rangle_\sigma} \text{ct}_1^{\langle y \rangle_\sigma}; r_{\text{id}}) \\ &= \text{DDLog}_{N,w}(((N+1)^x g^r)^{\langle y \cdot s \rangle_\sigma} (f^r)^{\langle y \rangle_\sigma}; r_{\text{id}}) \\ &= \text{DDLog}_{N,w}((N+1)^{\langle xy \rangle_\sigma} g^{\langle r \cdot y \rangle_\sigma} g^{\langle -r \cdot y \rangle_\sigma}; r_{\text{id}}) \\ &= \text{DDLog}_{N,w}((N+1)^{\langle xy \rangle_\sigma} g^{\langle 0 \rangle_\sigma}; r_{\text{id}}). \end{aligned}$$

From the correctness of DDLog and the fact that $xy < N^w 2^{-\tau}$ by our choice of parameters, $\langle xy \rangle'_\sigma$ is a random subtractive share of xy over the randomness of r' with probability at least $1 - 2^{-\tau}$.

By the pseudorandomness of the PRF, r_{id} can be replaced with truly random coins independent of the value of $r_1, \dots, r_{\text{id}-1}$ (which determines the distribution of the encryption randomness in $\{\{x\}\}$), so the correctness error is negligible.

We then show that the operation $\langle xy \rangle_\sigma \leftarrow \langle xy \rangle'_\sigma \bmod B^3 \cdot 2^{4\tau+3\lambda}$ preserves the correctness of the subtractive share $\langle xy \rangle_\sigma$. This follows with probability at least $1 - 2^{-\tau}$ because $xy < B^3 \cdot 2^{3\tau+3\lambda}$. We finally show that the operation $\langle xy \rangle_\sigma \leftarrow \langle xy \rangle'_\sigma \bmod B \cdot 2^\tau$ is correct. Let $B' = B \cdot 2^\tau$ be the modulus in the reduction. For $\sigma \in \{A, B\}$, $s_\sigma = t_\sigma B' + 1$, so $s_\sigma \equiv 1 \bmod B'$. This secret key structure carries over to the joint secret: $s = s_A s_B \equiv 1 \bmod B'$. It thus means that local reduction mod B' removes s , and is correct with probability at least $1 - 2^{-\tau}$ because that the RMS program bound dictates that $xy < B$. A formal explanation of why this is true can be found in Section 2.3.1 from the technical overview.

4. **Convert**: Follows trivially from the correctness of **Mult**.

5. **Output**: Follows trivially from the invariant that the memory shares are valid.

The correctness of **Output** immediately implies that the output of **DEval** is a correct subtractive share of $z = P(\mathbf{x})$, concluding the correctness proof for MKHSS.

Security. First, we remark that SEDL is known to be equivalent to SEI [ABC⁺24], so we will prove security from SEI. In addition, the security of the construction of NIM in Section 4.4 can be proven from DCR.

Recall that the security property requires that the input share $\llbracket x \rrbracket_{1-\sigma}^\sigma$ of party- $(1-\sigma)$, ensures the privacy of an input x shared using party- σ 's public key pk_σ .

Consider any efficient adversary $\mathcal{A}$ for the security experiment defined in Definition 11. Let the output of the security experiment be defined as 1 if $\mathcal{A}$'s output b' is equal to the challenge bit b ; else let the output of the experiment be defined as 0. We will use a hybrid argument to show that the output of the experiment is 1 with probability of at most $1/2 + \text{negl}(\lambda)$.

- *Hybrid $\mathcal{H}_0$* . This hybrid consists of the output of the experiment when run with adversary $\mathcal{A}$ when the challenge bit is $b = 0$.
- *Hybrid $\mathcal{H}_1$* . In this hybrid game, we replace $\text{pe}_A^{\text{nim}}, \text{pe}_B^{\text{nim}}$ inside pk_σ with encodings of zero. That is, we replace line 5 and 6 in MKHSS.KeyGen with $(\text{pe}_{\sigma'}^{\text{nim}}, -) \leftarrow \text{NIM.Encode}(\text{crs}^{\text{nim}}, \sigma', 0)$ for $\sigma' \in \{A, B\}$, respectively. Note that the security game discards $\text{sk}_{\sigma'}$ so $\text{st}_{\sigma'}^{\text{nim}}$ need not be simulated.

Claim. $\mathcal{H}_1 \approx_c \mathcal{H}_0$ assuming the security of NIM.

Proof. The claim follows immediately from the security of the NIM scheme. $\square$

- *Hybrid $\mathcal{H}_2$* . In this hybrid game, we replace the DJEG encryptions in $\llbracket x_0 \rrbracket_{1-\sigma}^\sigma$ of party- $(1-\sigma)$ with encryptions of x_1 .

Claim. $\mathcal{H}_2 \approx_c \mathcal{H}_1$ assuming DCR and SEI.

Proof. Notice that the DJEG encryptions in our MKHSS construction samples the secret key from the following distribution:

$$S_{\text{sk}} = \{t \cdot B \cdot 2^\tau + 1 : t \leftarrow_{\$} [2^{2\lambda}]\}.$$

We first show that the above distribution satisfies the condition in Equation (1). Recall that it stipulates that:

$$\mathcal{D}_0 = \{(g, g^s) : s \leftarrow \$ S_{\text{sk}}\} \approx_c \{(g, g') : g' \leftarrow \$ \mathbb{G}\} = \mathcal{D}_1,$$

where $\mathbb{G}$ is the group of $2N^w$ -th residues and g the generator of $\mathbb{G}$ in the common reference string. Let $B' = B \cdot 2^\tau$. With overwhelming probability, $\gcd(B', \phi(N)/4) = 1$, where $\phi(N)/4$ is the order of $\mathbb{G}$. Let g' denote a random element of $\mathbb{G}$. We proceed with a hybrid argument:

$$\begin{aligned} \mathcal{D}_0 &\approx_p (g, (g^t)^{B'} \cdot g) \\ &\approx_c (g, (g')^{B'} \cdot g) \quad (\text{By } \mathbb{G}\text{-SEI}) \\ &\approx_s (g, g' \cdot g) \quad (\text{By } \gcd(B', \phi(N)/4) = 1) \\ &\approx_s (g, g') \\ &\approx_p \mathcal{D}_1. \end{aligned} \tag{3}$$

This shows that Equation (1) is satisfied for our S_{sk} .

Then the claim follows by a straightforward hybrid argument replacing $\llbracket x \rrbracket_{1-\sigma}^\sigma := (\text{ct}_0, \text{ct}_1)$ encrypting x_0 with encryptions of x_1 and invoking the semantic security of our short exponent variant of the DJEG encryption scheme, proven in Lemma 5. $\square$

- *Hybrid $\mathcal{H}_3$.* In this hybrid game, we reverse the changes made in $\mathcal{H}_1$ and encode t using the NIM scheme.

Claim. $\mathcal{H}_3 \approx_c \mathcal{H}_2$ assuming the security of NIM.

Proof. The claim follows immediately from the security of the NIM scheme. $\square$

To complete the proof, observe that $\mathcal{H}_3$ is exactly the output of the experiment when the challenge bit $b = 1$. Since we've shown that $\mathcal{H}_0 \approx_c \mathcal{H}_3$, it follows that $\mathcal{A}$ wins the MKHSS security game with probability of at most $1/2 + \text{negl}(\lambda)$. $\blacksquare$

7 Full ANIKE Construction

We provide the full construction in Construction 6.

Theorem 15. *Assuming DCR, SEDL over the the group of $2N^w$ -th residues over $\mathbb{Z}_{N^{w+1}}^*$, and the existence of any NIKE scheme, Construction 6 is a correct and secure ANIKE scheme.*⁷

Proof. Correctness follows trivially from the correctness of MKHSS. For security, observe that key-pseudorandomness follows trivially from the CKS-light security [FHKP13] of NIKE since k_{nike} is hashed into the final key, and attribute-hiding follows trivially from the security of MKHSS. It remains to show the nontrivial property of key independence:

Suppose without the loss of generality that the key-independence adversary $\mathcal{A}$ wins the game with $\sigma = A$, we construct another adversary $\mathcal{A}'$ that breaks the multi-share (Remark 12) security of MKHSS for $\sigma = B$ by wrapping round $\mathcal{A}$'s input, output, and random oracle calls. Observe that for $\mathcal{A}$ to have any non-negligible advantage, it *must* query the random oracle on $\langle P(\mathbf{x}_A, \mathbf{x}_B) \cdot s \rangle_B = \langle s \rangle_B$ non-negligibly often, since otherwise k_0 and k_1 are identically distributed by the randomness of the oracle H .

First, $\mathcal{A}'$ relays crs and pass $\sigma = A$ to $\mathcal{A}$, who then responds with $\mathbf{x}_A, \mathbf{x}_B$. For $b \in \{0, 1\}$, $\mathcal{A}'$ sets $\mathbf{x}_b = (\mathbf{x}_B, b)$, and sends $\mathbf{x}_0, \mathbf{x}_1$ to the MKHSS challenger and get back $\mathbf{x}_b$ parsed out as $\llbracket \mathbf{x}_B \rrbracket_A^B, \llbracket b \rrbracket_A^B$. Then, $\mathcal{A}'$ runs Alice's protocol honestly, saves the coins r_A , and generates a NIKE keypair sk_B, pk_B for Bob. It then samples a *fake* Bob's key $k_b^* \leftarrow \$ \{0, 1\}$ and invokes $\mathcal{A}(k_b, (\text{pk}_B, \text{pe}, \llbracket \mathbf{x}_B \rrbracket_A^B), r_A, \text{state})$.

Finally, $\mathcal{A}'$ scans through the list of queries that $\mathcal{A}$ makes. For every query, it tries to parse it as $\langle s_B \rangle$, compute $s = \langle s \rangle_A - \langle s \rangle_B$, and check if $g^{-s} = f_B^{s_A}$. If so, it decrypts the DJEG ciphertext $\llbracket b \rrbracket_A^B$ and learns b . This must happen with non-negligible probability since up until the first query to the random oracle with $\langle s \rangle_B$, $\mathcal{A}$ cannot tell whether it is interacting with a real challenger or used as a subroutine of $\mathcal{A}'$ (specifically, the fake key k_b looks just like a challenger's key) due to the randomness of the oracle. $\blacksquare$

⁷ MKHSS implies NIKE, see [BGI⁺18].

Construction 6: Attribute-based non-interactive key exchange from NIKE and our MKHSS scheme.

Parameters. Let $\text{MKHSS} = (\text{Setup}, \text{KeyGen}, \text{Share}, \text{Eval}, \text{Eval}^*)$ be any MKHSS scheme with a modified Eval^* algorithm that outputs $\langle P(x_A, x_B) \cdot s \rangle_\sigma$. We can adapt the standard correctness definition and its security proof to easily show that our MKHSS scheme satisfies this, by observing that we can have a modified DEval^* algorithm whose output instruction produces $\langle zs \rangle_\sigma$. Let $\text{NIKE} = (\text{Setup}, \text{KeyDer}, \text{KeyGen})$ an arbitrary NIKE scheme (e.g., Diffie-Hellman). Let P be the fixed predicate for this instantiation of ANIKE.

ANIKE.Setup(1^λ):

```

1 :  $\text{crs}_{\text{mkhss}} \leftarrow \text{MKHSS.Setup}(\lambda)$ 
2 :  $\text{crs}_{\text{nike}} \leftarrow \text{NIKE.Setup}(\lambda)$ 
3 : return ( $\text{crs}_{\text{mkhss}}, \text{crs}_{\text{nike}}$ )

```

ANIKE.AttrKeyDer($\sigma, \text{st}_\sigma, \text{pe}_{1-\sigma}$):

```

1 : parse  $\text{pe}_{1-\sigma} = (\text{pk}, \text{pk}_{1-\sigma}^{\text{nike}}, \llbracket \mathbf{x}_{1-\sigma} \rrbracket_\sigma^{1-\sigma})$ 
2 : parse  $\text{st}_\sigma = (\text{sk}, \text{sk}_\sigma^{\text{nike}}, \llbracket \mathbf{x}_\sigma \rrbracket_\sigma^\sigma)$ 
3 :  $\langle P(\mathbf{x}_A, \mathbf{x}_B) \cdot s \rangle_\sigma := \text{MKHSS.Eval}^*(\text{crs}, \sigma, \text{sk}_\sigma, \text{pk}_{1-\sigma}, \llbracket \mathbf{x}_A \rrbracket_\sigma^A, \llbracket \mathbf{x}_B \rrbracket_\sigma^B, P)$ 
4 :  $k_{\text{nike}} := \text{NIKE.KeyDer}(\text{pk}_{1-\sigma}^{\text{nike}}, \text{sk}_\sigma^{\text{nike}})$ 
5 :  $k := H(\langle P(\mathbf{x}_A, \mathbf{x}_B) \cdot s \rangle_\sigma \| k_{\text{nike}} \| P)$ 
6 : return  $k$ 

```

ANIKE.AttrKeyGen($\text{crs}, \sigma, \mathbf{x}$):

```

1 : parse  $\text{crs} = (\text{crs}_{\text{mkhss}}, \text{crs}_{\text{nike}})$ 
2 :  $(\text{pk}, \text{sk}) \leftarrow \text{MKHSS.KeyGen}(\text{crs})$ 
3 :  $(\text{pk}^{\text{nike}}, \text{sk}^{\text{nike}}) \leftarrow \text{NIKE.KeyGen}(\text{crs}_{\text{nike}})$ 
4 :  $(\llbracket \mathbf{x} \rrbracket_A^\sigma, \llbracket \mathbf{x} \rrbracket_B^\sigma) \leftarrow \text{MKHSS.Share}(\text{crs}, \sigma, \text{pk}, \mathbf{x})$ 
5 :  $\text{pe} := (\text{pk}, \text{pk}^{\text{nike}}, \llbracket \mathbf{x} \rrbracket_{1-\sigma}^\sigma)$ 
6 :  $\text{st} := (\text{sk}, \text{sk}^{\text{nike}}, \llbracket \mathbf{x} \rrbracket_\sigma^\sigma)$ 
7 : return ( $\text{pe}, \text{st}$ )

```

8 Evaluation

We implemented in C++ our MKHSS scheme *in addition* to a naïve implementation of the Couteau et al. [CDH⁺25] construction from DCR, which we denote as the baseline.⁸ We provide empirical and analytical performance comparisons, using the experimental setup described in Section 8.1. We denote $\lg(\cdot)$ as the base-2 logarithm.

8.1 Experimental Setup

8.1.1 Implementation Details We use binomial coefficients to compute $(N+1)^x \bmod N^{w+1}$. This optimization is well-known in prior work [DJN10], so we adopt this optimization for both our implementation and the baseline.

Whenever modular exponentiation is repeated with the same base two or more times, we precompute powers of this base to speed up future operations. Specifically, we use either the fixed-base windowing or the fixed-base comb method [MvOV97], whichever is faster. Such a *fixed-base exponentiation* optimization is well-known [BCG⁺17], so we apply it to both our scheme and the baseline.

For MKHSS, we compute the joint public key $f = g^{-s_A s_B}$ once when initializing the RMS program, and reuse the same value when synchronizing all sender input shares. We apply this optimization to both our scheme and the baseline to ensure comparability between the schemes.

⁸ <https://github.com/kevin-he-01/mkhss/>

For ANIKE, we instantiate the internal NIKE scheme (Section 3.3) from an elliptic-curve Diffie-Hellman (ECDH) key exchange using the NIST P-256 curve.

8.1.2 Hardware We use the following two machines:

1. *Laptop*. This is an x86_64 Linux machine running Ubuntu 22.04 on an AMD Ryzen 7 7735U processor, equipped with 32 GiB of memory. It runs the experiments to compare our scheme against the baseline.
2. *AWS*. This is an AWS bare metal instance of type `c7i.metal-24x1`, an x86_64 Linux machine running Amazon Linux 2023 with support for the AVX512-IFMA instruction set. It runs the experiments to demonstrate speedup using an efficient vectorized modular multiplication algorithm [LD25].

We only use a single core for both machines.

8.1.3 Parameters We pick the computational and statistical security parameters to be $\lambda = 128$, $\tau = 128$ respectively.

The parameter B determines the largest integer (by absolute value) allowed during the computation for an RMS program. We vary $\lg(B) \in \{1, 512, 1024, 2048\}$ to show that our scheme scales to RMS programs that use integers ranging from 1 to 2048 bits long. We remark that all of our applications to key exchange only need $\lg(B) = 1$ because they compute on values in binary form.

For all values of B , we set l_N , the bit length of the modulus N , to be at least 3072 bits (possibly larger), which provides at least 128 bits of security against factoring attacks. In addition, recall from Section 2.4 that to ensure correctness, we need to ensure $wl_N \geq 3\log_2(B) + 4\lambda + 3\tau$, where $w \geq 1$ is an integer parameter that we can freely pick to optimize the scheme with no impact on security as long as the correctness condition holds. For large values of $\lg(B)$, we either increase l_N or the DJEG parameter w (See Section 4.2) in our scheme:

1. For $\lg(B) = 1024$, we set $w = 1$ and $l_N = 3968$.
2. For $\lg(B) = 2048$, we set $w = 2$ and $l_N = 3520$.
3. For all other values $\lg(B) \leq 512$, we set $w = 1$ and $l_N = 3072$ for our scheme.

We set $l_N = 3072$ and $w = 3$ for the baseline on all parameters, since correctness of the baseline will not break down for our choices of $\lg(B)$.

8.2 Multi-Key HSS

Throughout the evaluation, we refer to MKHSS procedures using descriptive names. In Table 1, we show the correspondence between descriptive names and formal names from the full construction (Construction 5).

For $\lg(B) = 1$, we compare the running time to complete each procedure in the MKHSS construction and each RMS program instruction in Table 1 using our *laptop* machine.

The numbers are reported for running each procedure without precomputed values for fixed-base exponentiation, modeling application scenarios where each procedure is only invoked once. See Section 8.4 for an application that takes significant advantage of fixed-base exponentiation.

Since the performance of our multi-key HSS scheme depends on the bound B , we vary $\lg(B)$ from 1 to 2048 to show the scalability of our scheme when computing on very large integers in Figure 4 and show consistent improvement over the baseline. The baseline supports all these parameter sets with the same running time.

Lastly, we compare the size of keys, input, and memory shares for varying $\lg(B)$ in Figure 5.

8.2.1 Analytical Performance Comparison To highlight our advantage over the baseline, we only consider parameter ranges where the correctness condition (Section 2.4) holds for $w = 1$. For our concrete parameter choices in Section 8.1.3, this means $\lg(B) \leq 725$.

We remark that a similar analysis can be done to show an advantage over the baseline for any $\lg(B)$ value where we only need a smaller w compared to the baseline (e.g., $w = 2$ vs $w = 3$ for

Table 1: The running time of procedures for our MKHSS scheme versus the baseline for $\lg(B) = 1$ on our *laptop*.

Procedure	Name in Construction 5	Ours	Baseline	Our speedup
Generate CRS*	MKHSS.Setup	28.5 ms	251.9 ms	$8.8\times$
Generate key pair	MKHSS.KeyGen	7.0 ms	104.5 ms	$15\times$
Encode an input share	MKHSS.Share	2.6 ms	10.3 ms	$4.0\times$
Init RMS program (Alice)	InitRMS ($\sigma = A$)	4.4 ms	96.4 ms	$22\times$
Init RMS program (Bob)	InitRMS ($\sigma = B$)	3.1 ms	50.3 ms	$16\times$
Sync sender input share	ExpLinEncS	1.3 ms	5.2 ms	$4.0\times$
Sync received input share	ExpLinEncR	1.8 ms	61.8 ms	$35\times$
Procedure	Name in Construction 5	Ours	Baseline	Our speedup
Add two inputs	DEval.ScaleAdd _I ($a = 1, b = 1$)	$9.5 \mu\text{s}$	$38.3 \mu\text{s}$	$4.0\times$
Subtract two inputs	DEval.ScaleAdd _I ($a = 1, b = -1$)	$67.3 \mu\text{s}$	$341.6 \mu\text{s}$	$5.1\times$
Add two memory values	DEval.ScaleAdd _M ($a = 1, b = 1$)	19.6 ns	63.8 ns	$3.3\times$
Multiply memory and input	DEval.Mult	5.0 ms	224.6 ms	$45\times$
Convert input to memory	DEval.Convert	4.2 ms	167.8 ms	$40\times$

*: Excluding the time to generate the two safe primes p and q .

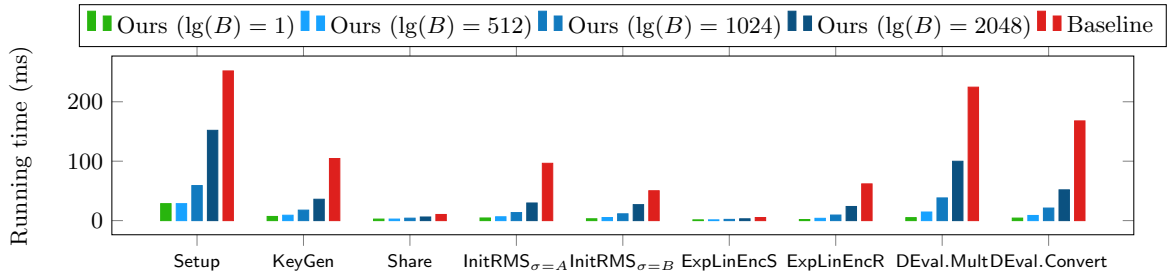


Fig. 4: The runtime of our scheme versus the baseline for various $\lg(B)$. See Table 1 for the meaning of labels.

Table 2: The analytical running time of significant procedures for this MKHSS scheme versus the baseline. We assume the Karatsuba multiplication exponent $e_{\text{mul}} = \lg(3)$, and let $T_{\text{exp}}(m) = 3 \cdot c_{\text{exp}} \cdot m^{\lg(3)}$.

Procedure	Analytical runtime		Speedup for $\lg(B) = 1$		Speedup for $\lg(B) = 512$	
	Ours	Baseline	Analytical	Empirical	Analytical	Empirical
Encode an input share	$4\lambda \cdot T_{\text{exp}}(l_N)$	$16\lambda \cdot T_{\text{exp}}(l_N)$	$4.0\times$	$4.0\times$	$4.0\times$	$4.0\times$
Sync sender input share	$2\lambda \cdot T_{\text{exp}}(l_N)$	$8\lambda \cdot T_{\text{exp}}(l_N)$	$4.0\times$	$4.0\times$	$4.0\times$	$4.0\times$
Sync received input share	$(\lg(B) + \tau + 2\lambda) \cdot T_{\text{exp}}(l_N)$	$4(l_N + 2\lambda) \cdot T_{\text{exp}}(l_N)$	$35\times$	$35\times$	$15\times$	$16\times$
Multiply memory and input	$(4\lg(B) + 5\tau + 3\lambda) \cdot T_{\text{exp}}(l_N)$	$16l_N T_{\text{exp}}(l_N)$	$48\times$	$45\times$	$16\times$	$16\times$

$\lg(B) = 1024$). In addition, it is reasonable to assume the integers are under 725 bits in practice. In fact, our applications to key exchange only require $\lg(B) = 1$.

Running Time. The most expensive procedures in both our scheme and the baseline are group operations, and especially modular exponentiations. Since the baseline works in the group $\mathbb{Z}_{N^4}^*$ ($w = 3$) while we work in $\mathbb{Z}_{N^2}^*$ ($w = 1$), merely counting the number of group multiplications or exponentia-

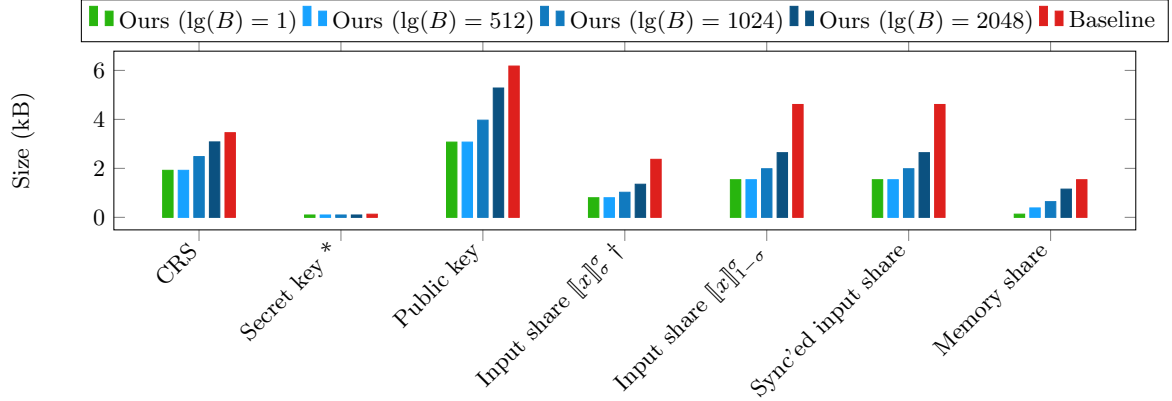


Fig. 5: The size of keys and encodings for our MKHSS scheme versus the baseline.

*: For both our scheme and the baseline, since the encryption secret is of the form $s = tX + 1$, we only store t to save space.

†: For the baseline, $[x]_\sigma^\dagger$ contains x in plaintext, so its size is dependent on $|x|$. We pick $|x| = 1$ to give the baseline an advantage.

Table 3: The quartiles of prime generation for various l_N .

l_N	First quartile	Second quartile (median)	Third quartile
3072	1.3 s	1.8 s	3.3 s
3520	1.6 s	3.6 s	6.6 s
3968	3.8 s	5.0 s	8.4 s

tions is not enough to give accurate performance estimates. We thus make a simplifying assumption, that group multiplication of m -bit numbers takes time $c_{\text{mul}} \cdot m^{e_{\text{mul}}}$ for some constants c_{mul} and e_{mul} (e.g., $e_{\text{mul}} = \lg(3) \approx 1.58$ for the Karatsuba [KO62] algorithm), and that modular exponentiations are implemented using a square-and-multiply-like algorithm, potentially using sliding window optimizations, whose time grows linearly in the length of the exponent (whose bits are roughly a uniform distribution over $\{0, 1\}$). This gives a concrete exponentiation cost estimate of $c_{\text{exp}} \cdot l \cdot m^{e_{\text{mul}}}$ where l is the exponent length in bits.

We present the analytical running time for each significant MKHSS procedure in Table 2. We include only group operations and prime generation time, and ignore symmetric-key operations since their cost is negligible. Whenever there is at least one modular exponentiation, we neglect the cost of a few extra group multiplication operations. The cost of exponentiation base- $(N + 1)$ is likewise negligible because of the optimization based on binomial expansion. We use $\lg$ to denote the base-2 logarithm.

8.2.2 Safe Prime Generation We require generating two safe primes as part of the procedure to generate a common reference string (CRS). We use an optimized implementation that trial divides both p and $(p - 1)/2$ before modular-exponentiation-based primality tests. Recall the CRS only needs to be generated once, and can be used by all future users of the protocol. For completeness, we run 10 trials for the procedure to generate an l_N -bit modulus as a product of two safe primes and report the quartiles in Table 3.

8.3 Application: Geolocation-Based Key Exchange

We report the performance of applying our MKHSS scheme to the geolocation-based key exchange below.

Table 4: Comparing runtime of geolocation-based key exchange using different MKHSS schemes on our *laptop*.

Procedure	$D = 2, L = 32$			$D = 3, L = 48$			$D = 4, L = 64$		
	Ours	Baseline	Speedup	Ours	Baseline	Speedup	Ours	Baseline	Speedup
GeoKE.Setup *	30.8 ms	258 ms	8.4×	29.2 ms	256 ms	8.8×	29.3 ms	256 ms	8.7×
GeoKE.AttrKeyGen($\sigma = A$)	64.0 ms	256 ms	4.0×	103 ms	391 ms	3.8×	163 ms	566 ms	3.5×
GeoKE.AttrKeyGen($\sigma = B$)	91.8 ms	355 ms	3.9×	166 ms	580 ms	3.5×	269 ms	902 ms	3.4×
GeoKE.AttrKeyDer($\sigma = A$)	1.56 s	53.7 s	34×	3.47 s	121 s	35×	6.19 s	216 s	35×
GeoKE.AttrKeyDer($\sigma = B$)	1.47 s	50.6 s	34×	3.24 s	113 s	35×	5.76 s	203 s	35×
A complete key exchange †	1.65 s	54.1 s	33×	3.63 s	122 s	33×	6.46 s	216 s	33×

*: Excludes prime generation time of MKHSS.

†: The longest time for any party to complete `AttrKeyGen` followed by the longest time for any party to complete `AttrKeyDer`. This is the total time to complete a key exchange excluding communication time. We exclude time for `Setup` because it is a one-time cost and the CRS can be reused.

Table 5: Communication (public encodings) and local storage (private state) requirement of a geolocation-based key exchange using our MKHSS scheme versus the baseline on our *laptop*.

Data	$D = 2, L = 32$			$D = 3, L = 48$			$D = 4, L = 64$		
	Ours	Baseline	Savings	Ours	Baseline	Savings	Ours	Baseline	Savings
Common reference string <code>crs</code>	1.9 kB	3.5 kB	1.8×	1.9 kB	3.5 kB	1.8×	1.9 kB	3.5 kB	1.8×
Alice’s public encoding <code>pe_A</code>	101.4 kB	301.1 kB	3.0×	224.3 kB	669.8 kB	3.0×	396.3 kB	1185.9 kB	3.0×
Bob’s public encoding <code>pe_B</code>	199.7 kB	596.0 kB	3.0×	445.5 kB	1333.3 kB	3.0×	789.5 kB	2365.5 kB	3.0×
Alice’s private state <code>st_A</code>	51.3 kB	151.8 kB	3.0×	115.3 kB	341.3 kB	3.0×	204.9 kB	606.6 kB	3.0×
Bob’s private state <code>st_B</code>	102.5 kB	303.4 kB	3.0×	230.5 kB	682.4 kB	3.0×	409.7 kB	1213.1 kB	3.0×
Total communication *	301.1 kB	897.2 kB	3.0×	669.8 kB	2003.1 kB	3.0×	1185.9 kB	3551.4 kB	3.0×

*: The sum of the sizes of `peA` and `peB`, which need to be communicated.

Parameters. The geolocation-based key exchange models a location x_σ as a D -dimensional coordinate of L -bit integers, and key exchange is successful if the distance between x_A and x_B is less than a threshold d under the Chebyshev distance metric.

We support arbitrary distance thresholds d , whose value does not affect our running time, so we pick d at random for evaluation.

Terminology. Recall that since our scheme is an instantiation of ANIKE, and we refer the reader to Figure 1 for the communication pattern of the ANIKE protocol. To summarize, each party publishes a public encoding based on their geolocation using `GeoKE.AttrKeyGen`, and the other party downloads the public encoding and runs `GeoKE.AttrKeyDer` to derive a key, and the two keys agree only if the locations are close. Each party needs to store some amount of private state in between the two calls.

The geolocation-based key exchange is asymmetric in the sense that one party (who can specify the distance threshold d) has a larger public encoding and a longer running time for `GeoKE.AttrKeyGen`. We denote this party as Bob in our evaluation.

Result. We present the running time for each procedure for various choices of D and L in Table 4. $D = 2$ and $L = 32$ gives a key exchange based on real-world geographic coordinates (i.e., latitude and longitude) at the precision of at least a centimeter. $D = 3$ allows incorporating elevation, and it can also be used to account for the curvature of the Earth over large distances. Lastly, $D = 4$ accounts for the possibility of additional numerical requirements for a policy, such as an acceptable price per night using the homestay example introduced in Section 3. Among all parameter regimes, we show a consistent speedup of at least $40\times$ over the baseline when used to instantiate this application.

Finally, the sizes of public encodings and private states, which determine communication and local storage requirements, are in Table 5. We show a consistent $3\times$ savings over the baseline.

8.4 Application: Fuzzy PAKE

We report the performance of applying our MKHSS scheme to fuzzy PAKE below.

Parameters. Recall that the parameter set for our fuzzy PAKE scheme is a 5-tuple (L, W, b, Q, T) :

1. L : The length of the passphrase.
2. W : The maximum length of a word in characters.
3. b : The number of bits per character.
4. Q : The number of character substitutions permitted in a word.
5. T : The number of incorrect words allowed.

Result. We provide the running time of `fPAKE.AttrKeyGen` and `fPAKE.AttrKeyDer` in Table 6 for a variety of parameters, and show a consistent speedup over the baseline:

1. $(L, W, b, Q, T) = (8, 9, 5, 2, 2)$, which is sufficient for a secure passphrase with words from Electronic Frontier Foundation’s large word list with appropriate filtering, see Section 8.4.1.
2. $(L, W, b, Q, T) = (10, 8, 16, 1, 1)$, which works with 10-word UTF-16 passphrases with up to 8-characters per word at a typo tolerance of 1 character per word and 1 incorrect word.
3. $(L, W, b, Q, T) = (12, 10, 8, 3, 3)$, which works with 12-word (extended) ASCII passphrases with up to 10-characters per word at a typo tolerance of 3 characters per word and 3 words.

Lastly, we give the sizes of public encodings and private states in Table 7.

8.4.1 Demonstrating practicality with real passphrases We show that the parameter set $(L, W, b, Q, T) = (8, 9, 5, 2, 2)$ provides sufficient security for a randomly chosen passphrase y under online attacks on a generic protocol as follows: An attacker gets to specify an arbitrary guess x , and the protocol responds with $P(x, y)$, which is 0 if the passphrases are approximately equal, and 1 otherwise.

We define the *fuzzy security level* of a passphrase distribution $\mathcal{D}$ as the negative base-2 logarithm of the maximum probability that any (adversarially chosen) passphrase x , not necessarily in the support of $\mathcal{D}$, matches a randomly generated passphrase $y \leftarrow \mathcal{D}$ according to our predicate:

$$-\log_2 \left(\max_{x \in (\Sigma \cup \{\perp\})^{LW}} \left\{ \Pr_{y \leftarrow \mathcal{D}} [P(x, y) = 0] \right\} \right).$$

Table 6: The running time of fuzzy PAKE using our MKHSS scheme versus the baseline on our *laptop*.

Procedure	$(L, W, b, Q, T) = (8, 9, 5, 2, 2)$			$(L, W, b, Q, T) = (10, 8, 16, 1, 1)$			$(L, W, b, Q, T) = (12, 10, 8, 3, 3)$		
	Ours	Baseline	Speedup	Ours	Baseline	Speedup	Ours	Baseline	Speedup
fPAKE.Setup	30.4 ms	270 ms	8.9×	31.1 ms	257 ms	8.3×	30.9 ms	260 ms	8.4×
fPAKE.AttrKeyGen($\sigma = A$)	214 ms	700 ms	3.3×	574 ms	1.70 s	3.0×	455 ms	1.37 s	3.0×
fPAKE.AttrKeyGen($\sigma = B$)	212 ms	671 ms	3.2×	557 ms	1.69 s	3.0×	450 ms	1.35 s	3.0×
fPAKE.AttrKeyDer($\sigma = A$)	7.35 s	252 s	34×	19.1 s	675 s	35×	27.0 s	919 s	34×
fPAKE.AttrKeyDer($\sigma = B$)	7.27 s	251 s	35×	19.2 s	676 s	35×	27.0 s	917 s	34×
A complete key exchange	7.56 s	252 s	33×	19.7 s	678 s	34×	27.5 s	920 s	33×

Table 7: Communication and local storage requirement of a fuzzy PAKE using our MKHSS scheme versus the baseline on our *laptop*. Since PAKE is symmetric, the results are the same for both Alice and Bob.

Data	$(L, W, b, Q, T) = (8, 9, 5, 2, 2)$			$(L, W, b, Q, T) = (10, 8, 16, 1, 1)$			$(L, W, b, Q, T) = (12, 10, 8, 3, 3)$		
	Ours	Baseline	Savings	Ours	Baseline	Savings	Ours	Baseline	Savings
Common reference string crs	1.9 kB	3.5 kB	1.8×	1.9 kB	3.5 kB	1.8×	1.9 kB	3.5 kB	1.8×
Public encoding pe	556.1 kB	1665.1 kB	3.0×	1969.2 kB	5904.4 kB	3.0×	1477.7 kB	4429.9 kB	3.0×
Private state st	556.1 kB	1665.1 kB	3.0×	1969.2 kB	5904.4 kB	3.0×	1477.7 kB	4429.9 kB	3.0×
Total communication	1.1 MB	3.3 MB	3.0×	3.9 MB	11.8 MB	3.0×	3.0 MB	8.9 MB	3.0×

We evaluated the fuzzy security level of a random passphrase where the eight words of the passphrase are chosen independently from a real-world wordlist, constructed as follows to minimize security loss due to fuzziness:

1. We started with Electronic Frontier Foundation’s large wordlist [Bon16], which contains 7776 words.⁹ There are 27 unique characters (lowercase a–z and the hyphen), so they fit in a 5-bit encoding even after accounting for the length-padding symbol $\perp$.
2. We filtered out all words with more than 10 neighbors (wordlist words within a Hamming distance of $Q = 2$).
3. We performed additional filtering to ensure that no word *outside* of the wordlist (i.e., arbitrary strings from $(\Sigma \cup \{\perp\})^W$) has more than 10 neighbors in the wordlist. We retain 6403 words, i.e., 82.3% of the original wordlist, so the security gain outweighs the slight security loss. We do this with a greedy algorithm to minimize the number of words to remove.

Using this wordlist, our parameters provide a fuzzy security level of 51.1 bits, which is comparable to the entropy (the security level) of a four-word passphrase with words from the entire EFF’s large wordlist under *exact matching*, which is 51.7 bits.

Remarks. While an eight-word passphrase is considered longer than typical standards, our fuzziness means that the user is only required to remember six out of eight words, in addition to typos within each word. In addition, the reduction of security is a function of the fuzzy predicate P alone. Our ANIKE protocol introduces no additional leakage, thus the adversary’s guessing advantage is the same as given by this protocol-independent analysis. Lastly, we constructed this wordlist solely for the purpose of demonstrating that our evaluation parameters provide adequate security: our fuzzy PAKE protocol does not assume that its input comes from any particular wordlist. Its fuzziness comes from the homomorphic computation of P rather than local typo corrections.

Table 8: The running time of geolocation-based key exchange using AVX512-accelerated modular multiplication by Langowski and Devadas [LD25] versus just using FLINT [tea25] on our *AWS* machine, using our optimized MKHSS scheme.

Procedure	$D = 2, L = 32$			$D = 3, L = 48$			$D = 4, L = 64$		
	AVX512	FLINT	Speedup	AVX512	FLINT	Speedup	AVX512	FLINT	Speedup
GeoKE.Setup	16.6 ms	55.9 ms	3.4×	16.8 ms	55.8 ms	3.3×	17.2 ms	55.8 ms	3.2×
GeoKE.AttrKeyGen($\sigma = A$)	24.5 ms	78.1 ms	3.2×	42.7 ms	130 ms	3.0×	66.6 ms	204 ms	3.1×
GeoKE.AttrKeyGen($\sigma = B$)	37.8 ms	119 ms	3.2×	68.7 ms	210 ms	3.1×	110 ms	336 ms	3.1×
GeoKE.AttrKeyDer($\sigma = A$)	660 ms	2.27 s	3.4×	1.50 s	5.08 s	3.4×	2.66 s	9.02 s	3.4×
GeoKE.AttrKeyDer($\sigma = B$)	609 ms	2.08 s	3.4×	1.38 s	4.64 s	3.4×	2.46 s	8.24 s	3.3×
A complete key exchange	698 ms	2.39 s	3.4×	1.57 s	5.29 s	3.4×	2.77 s	9.35 s	3.4×

Table 9: Similar AVX512 versus FLINT comparison (in the same manner as Table 8) for fuzzy PAKE.

Procedure	$(L, W, b, Q, T) = (8, 9, 5, 2, 2)$			$(L, W, b, Q, T) = (10, 8, 16, 1, 1)$			$(L, W, b, Q, T) = (12, 10, 8, 3, 3)$		
	AVX512	FLINT	Speedup	AVX512	FLINT	Speedup	AVX512	FLINT	Speedup
fPAKE.Setup	16.6 ms	55.6 ms	3.4×	16.9 ms	55.6 ms	3.3×	16.6 ms	55.7 ms	3.4×
fPAKE.AttrKeyGen($\sigma = A$)	82.8 ms	255 ms	3.1×	227 ms	669 ms	2.9×	182 ms	537 ms	2.9×
fPAKE.AttrKeyGen($\sigma = B$)	81.1 ms	251 ms	3.1×	222 ms	668 ms	3.0×	179 ms	537 ms	3.0×
fPAKE.AttrKeyDer($\sigma = A$)	3.09 s	10.0 s	3.2×	8.24 s	26.7 s	3.2×	11.5 s	36.6 s	3.2×
fPAKE.AttrKeyDer($\sigma = B$)	3.09 s	9.99 s	3.2×	8.21 s	26.7 s	3.2×	11.5 s	36.5 s	3.2×
A complete key exchange	3.17 s	10.3 s	3.2×	8.47 s	27.4 s	3.2×	11.7 s	37.2 s	3.2×

8.5 Further Acceleration Using Vectorization

Using efficient vectorized modular multiplication techniques [LD25], our scheme can be further accelerated on machines that support the AVX512-IFMA instruction set. We compare the running times of our two key exchange applications on the *AWS* machine with and *without* the use of Langowski and Devadas [LD25] in Table 8 and Table 9.

9 Related Works

Both multi-key HSS and attribute-based non-interactive key exchange are new notions that distinguish themselves from prior work in many aspects. We compare with prior work to further highlight the differences.

Drawbacks of weaker notions of HSS. We stress that this is the first practical implementation of *multi-key* HSS. All prior constructions achieve at best the weaker notion of public-key HSS [BGI17], which cannot be used to cleanly instantiate our applications to key exchange without resorting to extra trust assumptions, the need for a correlated setup, and extra rounds of communication.

Additional differences from Boyle et al. Boyle et al. [BCG⁺17] provided a concrete implementation of public-key HSS with worse security guarantees. Unlike our work, they have a high and non-negligible failure probability (on the order of 2^{-5} to 2^{-15} for practical parameters), which leaks inputs and keys via a selective-failure attack. In addition, they evaluated on an implementation that only achieves 80 bits of security (while we evaluate on 128 bits, the standard level to defend against all realistic adversaries). Despite their efforts to reduce the leakage via leakage-absorbing pads, they (and all known constructions based on DDH) cannot circumvent the barrier [DKK18] that a failure probability under ε requires blowing up computation to $\Omega(1/\sqrt{\varepsilon})$.

Alternative Approaches to Key Exchange. To the best of our knowledge, all prior approaches to attribute-based key exchange require interaction [DHP⁺18, KKL⁺16, BFH⁺23], i.e., at least one party needs to receive a message from the other party *before* it can encode its own inputs. In addition to performance overheads of communication, interactivity complicates the security proof, and many schemes (e.g., [DHP⁺18]) are broken due to subtle flaws [JRX25].

One further difference with [KKL⁺16] is that they require a trusted authority to vouch for each client’s input attributes. In contrast, ANIKE only requires a single reusable CRS generated in any trusted manner, including via a secure multi-party computation. In addition, our inputs can be self-reported, which is especially relevant for sensitive information like passphrases and travel destinations.

10 Conclusion

We presented a concretely-efficient construction of multi-key homomorphic secret sharing, a recently introduced primitive that can be applied to a rich class of attribute-based non-interactive key exchange protocols. Unlike prior approaches to HSS and key exchange, MKHSS and ANIKE allows both parties to encode their inputs immediately without prior messages from another party. We believe our algorithmic insights that made our scheme practical may be of independent interest in single-key HSS as well. We identified several practically useful scenarios for key exchange, including the use of geolocation and passphrases, and believe that our work serves to bring HSS to practical realms.

Acknowledgments

We would like to thank Simon Langowski for helpful discussions around hardware acceleration using his algorithm [LD25] and implementation, and anonymous reviewers as well as the shepherd for their valuable comments. Kaiwen He and Srinivas Devadas acknowledge the support of the National Science Foundation (NSF). This material is based upon work supported by the NSF under Award No. 2330065. Geoffroy Couteau acknowledges the support of the French Agence Nationale de la Recherche (ANR), under the France 2030 ANR Project ANR-22-PECY003 SecureCompute. This work is supported by ERC grant OBELiSC (101115790).

⁹ https://www.eff.org/files/2016/07/18/eff_large_wordlist.txt

References

- ABC⁺24. M. Adjedj, C. Blokh, G. Couteau, A. Galansky, A. Joux, and N. Makriyannis. Two-round 2PC ECDSA at the cost of 1 OLE. *Cryptology ePrint Archive*, Report 2024/1950, 2024.
- ADOS22. D. Abram, I. Damgård, C. Orlandi, and P. Scholl. An algebraic framework for silent preprocessing with trustless setup and active security. In *Advances in Cryptology – CRYPTO 2022, Part IV, Lecture Notes in Computer Science* 13510, pages 421–452, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland.
- BCG⁺17. E. Boyle, G. Couteau, N. Gilboa, Y. Ishai, and M. Orrù. Homomorphic secret sharing: Optimizations and applications. In *ACM CCS 2017: 24th Conference on Computer and Communications Security*, pages 2105–2122, Dallas, TX, USA, October 31 – November 2, 2017. ACM Press.
- BCP03. E. Bresson, D. Catalano, and D. Pointcheval. A simple public-key cryptosystem with a double trapdoor decryption mechanism and its applications. In *Advances in Cryptology – ASIACRYPT 2003, Lecture Notes in Computer Science* 2894, pages 37–54, Taipei, Taiwan, November 30 – December 4, 2003. Springer Berlin Heidelberg, Germany.
- BDSS25. E. Boyle, L. Devadas, and S. Servan-Schreiber. Non-interactive distributed point functions. In *PKC 2025: 28th International Conference on Theory and Practice of Public Key Cryptography, Part I, Lecture Notes in Computer Science* 15674, pages 3–35, Røros, Norway, May 12–15, 2025. Springer, Cham, Switzerland.
- BFH⁺23. J. Bootle, S. H. Faller, J. Hesse, K. Hostáková, and J. Ottenhues. Generalized fuzzy password-authenticated key exchange from error correcting codes. In *Advances in Cryptology – ASIACRYPT 2023, Part VIII, Lecture Notes in Computer Science* 14445, pages 110–142, Guangzhou, China, December 4–8, 2023. Springer, Singapore, Singapore.
- BGI16. E. Boyle, N. Gilboa, and Y. Ishai. Breaking the circuit size barrier for secure computation under DDH. In *Advances in Cryptology – CRYPTO 2016, Part I, Lecture Notes in Computer Science* 9814, pages 509–539, Santa Barbara, CA, USA, August 14–18, 2016. Springer Berlin Heidelberg, Germany.
- BGI17. E. Boyle, N. Gilboa, and Y. Ishai. Group-based secure computation: Optimizing rounds, communication, and computation. In *Advances in Cryptology – EUROCRYPT 2017, Part II, Lecture Notes in Computer Science* 10211, pages 163–193, Paris, France, April 30 – May 4, 2017. Springer, Cham, Switzerland.
- BGI⁺18. E. Boyle, N. Gilboa, Y. Ishai, H. Lin, and S. Tessaro. Foundations of homomorphic secret sharing. In *ITCS 2018: 9th Innovations in Theoretical Computer Science Conference*, pages 21:1–21:21, Cambridge, MA, USA, January 11–14, 2018. Leibniz International Proceedings in Informatics (LIPIcs).
- BM92. S. M. Bellovin and M. Merritt. Encrypted key exchange: Password-based protocols secure against dictionary attacks. In *1992 IEEE Symposium on Security and Privacy*, pages 72–84. IEEE Computer Society Press, May 1992.
- Bon16. J. Bonneau. Deep dive: EFF’s new wordlists for random passphrases. *Electronic Frontier Foundation*, 2016.
- BPR00. M. Bellare, D. Pointcheval, and P. Rogaway. Authenticated key exchange secure against dictionary attacks. In *Advances in Cryptology – EUROCRYPT 2000, Lecture Notes in Computer Science* 1807, pages 139–155, Bruges, Belgium, May 14–18, 2000. Springer Berlin Heidelberg, Germany.
- CAA⁺16. R. Chatterjee, A. Athayle, D. Akhawe, A. Juels, and T. Ristenpart. pASSWORD tYPOS and how to correct them securely. In *2016 IEEE Symposium on Security and Privacy*, pages 799–818, San Jose, CA, USA, May 22–26, 2016. IEEE Computer Society Press.
- CDH⁺25. G. Couteau, L. Devadas, A. Hegde, A. Jain, and S. Servan-Schreiber. Multi-key homomorphic secret sharing. In *Advances in Cryptology – EUROCRYPT 2025, Part V, Lecture Notes in Computer Science* 15605, pages 3–33, Madrid, Spain, May 4–8, 2025. Springer, Cham, Switzerland.
- CGKR22. G. Couteau, D. Goudarzi, M. Klooß, and M. Reichle. Sharp: Short relaxed range proofs. In *ACM CCS 2022: 29th Conference on Computer and Communications Security*, pages 609–622, Los Angeles, CA, USA, November 7–11, 2022. ACM Press.
- CKLR21. G. Couteau, M. Klooß, H. Lin, and M. Reichle. Efficient range proofs with transparent setup from bounded integer commitments. In *Advances in Cryptology – EUROCRYPT 2021, Part III, Lecture Notes in Computer Science* 12698, pages 247–277, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland.
- DH76. W. Diffie and M. E. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976.
- DHP⁺18. P.-A. Dupont, J. Hesse, D. Pointcheval, L. Reyzin, and S. Yakubov. Fuzzy password-authenticated key exchange. In *Advances in Cryptology – EUROCRYPT 2018, Part III, Lecture Notes in Computer Science* 10822, pages 393–424, Tel Aviv, Israel, April 29 – May 3, 2018. Springer, Cham, Switzerland.

- DJ01. I. Damgård and M. Jurik. A generalisation, a simplification and some applications of Paillier’s probabilistic public-key system. In *PKC 2001: 4th International Workshop on Theory and Practice in Public Key Cryptography, Lecture Notes in Computer Science* 1992, pages 119–136, Cheju Island, South Korea, February 13–15, 2001. Springer Berlin Heidelberg, Germany.
- DJ03. I. Damgård and M. Jurik. A length-flexible threshold cryptosystem with applications. In *ACISP 03: 8th Australasian Conference on Information Security and Privacy, Lecture Notes in Computer Science* 2727, pages 350–364, Wollongong, NSW, Australia, July 9–11, 2003. Springer Berlin Heidelberg, Germany.
- DJN10. I. Damgård, M. Jurik, and J. B. Nielsen. A generalization of Paillier’s public-key system with applications to electronic voting. *International Journal of Information Security*, 9:371–385, 2010.
- DKK18. I. Dinur, N. Keller, and O. Klein. An optimal distributed discrete log protocol with applications to homomorphic secret sharing. In *Advances in Cryptology – CRYPTO 2018, Part III, Lecture Notes in Computer Science* 10993, pages 213–242, Santa Barbara, CA, USA, August 19–23, 2018. Springer, Cham, Switzerland.
- DZX⁺24. W. Deng, B. Zhao, Y. Xiao, Y. Zhong, Q. Pei, and X. Liu. MORSE: An efficient homomorphic secret sharing scheme enabling non-linear operation. *arXiv preprint arXiv:2410.06514*, 2024.
- FHKP13. E. S. V. Freire, D. Hofheinz, E. Kiltz, and K. G. Paterson. Non-interactive key exchange. In *PKC 2013: 16th International Conference on Theory and Practice of Public Key Cryptography, Lecture Notes in Computer Science* 7778, pages 254–271, Nara, Japan, February 26 – March 1, 2013. Springer Berlin Heidelberg, Germany.
- FPS06. M. Friedl, N. Provos, and W. A. Simpson. Diffie-Hellman Group Exchange for the Secure Shell (SSH) Transport Layer Protocol. RFC 4419, March 2006.
- Gen09. C. Gentry. Fully homomorphic encryption using ideal lattices. In *41st Annual ACM Symposium on Theory of Computing*, pages 169–178, Bethesda, MD, USA, May 31 – June 2, 2009. ACM Press.
- Gil16. D. K. Gillmor. Negotiated Finite Field Diffie-Hellman Ephemeral Parameters for Transport Layer Security (TLS). RFC 7919, August 2016.
- HLP11. S. Halevi, Y. Lindell, and B. Pinkas. Secure computation on the web: Computing without simultaneous interaction. In *Advances in Cryptology – CRYPTO 2011, Lecture Notes in Computer Science* 6841, pages 132–150, Santa Barbara, CA, USA, August 14–18, 2011. Springer Berlin Heidelberg, Germany.
- JRX25. J. Januzelli, L. Roy, and J. Xu. Under what conditions is encrypted key exchange actually secure? In *Advances in Cryptology – EUROCRYPT 2025, Part II, Lecture Notes in Computer Science* 15602, pages 451–481, Madrid, Spain, May 4–8, 2025. Springer, Cham, Switzerland.
- KK04. T. Koshihara and K. Kurosawa. Short exponent Diffie-Hellman problems. In *PKC 2004: 7th International Workshop on Theory and Practice in Public Key Cryptography, Lecture Notes in Computer Science* 2947, pages 173–186, Singapore, March 1–4, 2004. Springer Berlin Heidelberg, Germany.
- KKL⁺16. V. Kolesnikov, H. Krawczyk, Y. Lindell, A. J. Malozemoff, and T. Rabin. Attribute-based key exchange with general policies. In *ACM CCS 2016: 23rd Conference on Computer and Communications Security*, pages 1451–1463, Vienna, Austria, October 24–28, 2016. ACM Press.
- KO62. A. A. Karatsuba and Y. P. Ofman. Multiplication of many-digit numbers by automatic computers. In *Doklady Akademii Nauk*, pages 293–294. Russian Academy of Sciences, 1962.
- LD25. S. Langowski and S. Devadas. Efficient modular multiplication using vector instructions on commodity hardware. Cryptology ePrint Archive, Report 2025/1068, 2025.
- MvOV97. A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone. *Handbook of Applied Cryptography*. The CRC Press series on discrete mathematics and its applications. CRC Press, 2000 N.W. Corporate Blvd., Boca Raton, FL 33431-9868, USA, 1997.
- OSY21. C. Orlandi, P. Scholl, and S. Yakubov. The rise of paillier: Homomorphic secret sharing and public-key silent OT. In *Advances in Cryptology – EUROCRYPT 2021, Part I, Lecture Notes in Computer Science* 12696, pages 678–708, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland.
- Pai99. P. Paillier. Public-key cryptosystems based on composite degree residuosity classes. In *Advances in Cryptology – EUROCRYPT’99, Lecture Notes in Computer Science* 1592, pages 223–238, Prague, Czech Republic, May 2–6, 1999. Springer Berlin Heidelberg, Germany.
- Pol78. J. M. Pollard. Monte carlo methods for index computation (mod p). *Mathematics of computation*, 32(143):918–924, 1978.
- Pro15. E. Protalinski. Facebook passwords are not case sensitive. <http://www.zdnet.com/article/facebook-passwords-are-not-case-sensitive-update/>, 2015. Accessed: 2025-09-28.
- RS21. L. Roy and J. Singh. Large message homomorphic secret sharing from DCR and applications. In *Advances in Cryptology – CRYPTO 2021, Part III, Lecture Notes in Computer Science* 12827, pages 687–717, Virtual Event, August 16–20, 2021. Springer, Cham, Switzerland.
- tea25. T. F. team. *FLINT: Fast Library for Number Theory*, 2025. Version 3.2.2, <https://flintlib.org>.

- VAS⁺17. L. Valenta, D. Adrian, A. Sanso, S. Cohnsey, J. Fried, M. Hastings, J. A. Halderman, and N. Heninger. Measuring small subgroup attacks against Diffie-Hellman. In *ISOC Network and Distributed System Security Symposium – NDSS 2017*, San Diego, CA, USA, February 26 – March 1, 2017. The Internet Society.
- XLX⁺24. Z. Xie, M. Liu, H. Xue, M. H. Au, R. H. Deng, and S.-M. Yiu. Direct range proofs for paillier cryptosystem and their applications. In *ACM CCS 2024: 31st Conference on Computer and Communications Security*, pages 899–913, Salt Lake City, UT, USA, October 14–18, 2024. ACM Press.

Supplementary Material

A Malicious Security

Our ANIKE construction achieves security against semi-honest adversaries, as reflected in the security experiment $\mathbf{E}_A^{\text{KI}}$.

Our ANIKE construction achieves security against semi-honest adversaries, as reflected in the security experiment $\mathbf{E}_A^{\text{KI}}$. It is easy to upgrade to semi-malicious security where the adversary can choose its coin r_σ in the game, by including $H(\text{pe}_\sigma)$ in each party's private state st_σ , and change line 4 of `InitRMS` in Construction 5 to derive the PRF key as $k_{\text{prf}} = H(H(\text{pe}_A) \| H(\text{pe}_B))$.

To achieve full malicious security, each party then appends a non-interactive zero knowledge proofs (NIZK) that the public encodings are valid, i.e., there exist some random coins r and valid B -bounded inputs $\mathbf{x}$ that will produce the encodings. A concretely efficient NIZK can be obtained via a Fiat-Shamir transform on generic sigma protocols over the ciphertext group $\mathbb{Z}_{N^{w+1}}$. For example, we can use a range proof [XLX⁺24] to ensure that $\mathbf{x}$ are B -bounded and that the secret key s_σ is of the right special form. Equality of discrete log proofs will also be useful to ensure the same secret is used in all the ciphertexts as well as the NIM encoding.

Lastly, we gain an additional advantage due to the non-interactivity of our key exchange protocol: the proofs over the public encodings can be verified by anyone, including parties who do not participate in the key exchange. We believe that this public verifiability property will deter cheating, even if not all parties verify the proof.